

Resumen INSW1

Bander

Actualizado por ult. vez: 6/3/2025

1 Introduccion

Resumen INSW1 en proceso(?)

2 Parte Teorica

2.1 Refactorings que se utilizan:

Un refactoring es un cambio a nivel disenio que no modifica el resultado de ejecucion.

- **Extract method:**
- **Rename:**
- **Insert Superclass:**
- **Change of Superclass:**
- **Push up:** Cambia el metodo del lugar en el que esta a su superclase.
- **Push down:** Cambia el metodo de donde esta a una subclase o todas.

2.2 SacarCodigoRepetido

1. Mover lo repetido a una nueva abstraccion.
2. Parametrizar lo que cambia
3. Poner nombre a la nueva abstraccion (**lo mas importante**).
4. Reemplazar el codigo repetido por uso de nueva abstraccion.

2.3 Reemplazar if

1. Crear una jerarquia polimorfica con una abstraccion para cada condicion de if (opcional).
2. Mover (contextualmente) el cuerpo de cada if a cada aabstraccion correspondiente usando el mismo mensaje.
3. Nombrar las abstracciones de **1**.
4. Nombrar el mensaje de **2**.
5. Reemplazar ifs por envio de mensajes polimorfico.
6. Buscar objeto polimorfico (opcional).

2.4 Heurísticas de diseño (reglas de manera contextual)

1. Crear objetos completos (desde el momento en que existe representa un objeto de la realidad). Por ejemplo, las instancias de las clases deben ser representantes de algo de la realidad ni bien existe.
2. Crear objetos validos (si no es valido, debe fallar en seguida).
3. Favorecer el uso de objetos inmutables (objetos que no se pueden modificar una vez instanciados) (solo se hacen objetos inmutables cuando lo que representan es inmutable) (me despreocupo del paso del tiempo (**transparencia referencial**)).
4. No usar nil/null (porque significa muchas cosas y no sigue la idea de disenio de que la funcion de relacion entre realidad y nuestro modelo es biyectiva) (**Hay un patron de disenio para sacar null**).
5. Evitar tener getters y setters a no ser que sea realmente necesario (esto es para evitar el romper encapsulamiento). Si el objeto a devolver es inmutable es inmutable, no hay problema, si no, es mejor devolver una copia (hacer object copy) (o si es muy grande devolver **ReadStream on:** objeto, qu edevuelve un string que no permite modificar el objeto, si no, solo leerlo).
6. Modificaciones atomicas.

2.5 ¿Como se hace TDD? (Test-Driven Development)

1. Escribir un test
 - Debe ser el mas sencillo que se nos ocurra.
 - Debe **fallar** al correrlo (si no es porque esta repetido o me adelanto al paso 2).
2. Correr todos los tests
 - Implementar **la solucion mas simple** que haga pasar el/los test/s
 - Volver al paso 2 hasta que todos los tests pasen.
3. Reflexiono (porque en el paso 2 se puede haber hecho chanchadas) - ¿Se puede mejorar el codigo?
 - Si -¿ Refactorizar y volver al paso 2
 - No -¿ Avanzar y volver al paso 1 con otro test que pensar.

3 Papers de primer parcial

3.1 Programming as Theory Building (by Peter Naur)

- **Ryle's Notion Theory:**

- Lo que caracteriza una actividad intelectual mas alla de si la actividad es simplemente inteligente, es la persona armando y teniendo una teoria, donde la misma es entendida como el conocimiento que una persona debe tener para no solo hacer ciertas acciones inteligentemente pero ademas poder explicarlas, poder responder preguntas sobre estas, tener debates y demas.

- **The thory to be built by the programmer:**

- El programador construye una teoria de como ciertos aspectos/temas (*affairs*) del mundo se pueden manejar/ser apoyados (*handeled*) por un programa.
- El conocimiento del programador supera a la documentacion al menos en 3 areas escenciales:
 1. Puede explicar como la solucion se relaciona con los asuntos del mundo que ayuda a manejar.
 2. Puede justificar cada parte del codigo.
 3. Puede responder constructivamente cualquier demanda de modificacion del programa para ayudar en los asuntos del mundo en una nueva manera.

- **Problems and Costs of Program Modifications:**

- Viendolo desde la perspectiva de Theory Building no se cree que sea generalmente posible que las modificaciones de un programa sean de bajo costo.
- La flexibilidad de un programa se puede conseguir en general con un costo substancial (ya que la usabilidad de esta depende unicamente del futuro).
- Para la modificacion de un programa una solucion ya programada debe ser modificada, el primer paso es comparar la solucion existente con las demandas generadas por las modificaciones deseadas. La similitud que debe ser reconocida es accesible por las personas que posean la teoria del programa.
- Si las modificaciones de un programa se viese solo como program text y su comportamiento externo de ejecucion, una modificacion podria ser realizada de varias maneras, todas correctas. Si se ve en relacion a la Theory of the program estas mismas modificaciones se podrian ver muy distintas.
- El character de los cambios hechos en un programa es fundamental para la viabilidad de este a largo plazo. Para que el programa mantenga su calidad es importante que cada modificacion este enlazada/fundamentada (*firmlly grounded*) en la teoria del mismo.
- Los problemas de la modificacion de un programa surgen de la suncion que programar consiste en texto del programa en vez de la actividad de theory building.

- **Program Life, Death, and Revival:**

- La teoria de los programas no se puede expresar, esta estrictamente relacionado con los seres humanos.
- El armado de un programa es lo mismo que el armado de la teoria del mismo.
- La muerte de un programa es cuando el equipo de programadores que tienen la teoria se disuelve. Esta se ve cuando las demandas de modificaciones no pueden ser resueltas inteligentemente.
- La resurreccion de un programa es la reconstruccion de su teoria por un nuevo equipo de programadores. Esto se debe intentar en casos excepcionales, pero generalmente conviene que se descarte el programa existente y que el equipo de programadores lo resuelva al problema de nuevo.
- Para que un nuevo programador obtenga una teoria de un programa existente debe ser instruido por alguien que posea ya la teoria del mismo y trabajar a su lado discutiendo la relacion del programa y el mundo, y los limites puestos en el mundo para el manejo del programa.

- **Methods and Thory Building:**

- **Programming method:** Conjunto de reglas de trabajo, que tipo de cosas los programadores deberian hacer, en que orden, que notaciones o lenguajes usar y que tipos de documentos producir en diversas etapas.

- En la construcción de la teoría no puede haber una particular secuencia de acciones.
- En la Theory Building view no hay un método concreto.
- En donde la theory building view difiere de las metodologías es en que técnicas usar y en que orden. En la primera depende exclusivamente en el programador para decidirlo tomando en cuenta el problema a ser resuelto.

- **Programmers' Status and the Theory Building View:**

- Theory building view choca más con las áreas de la contribución personal de un programador y el estatus de un programador.
- Mientras que en estas áreas el programador se ve como un componente de una máquina en Theory Building View el programador se ve como un desarrollador y administrador responsable de la actividad donde la computadora es una parte.
- Mientras que las notaciones, representaciones de datos, y procesamiento de datos sigue siendo de gran importancia, se debe hacer un énfasis principal en el entendimiento y talento de armado de teorías.

- **Conclusions:**

- Accepting program modifications demanded by changing external circumstances to be an essential part of programming, it is argued that the primary aim of programming is to have the programmers build a theory of the way the matters at hand may be supported by the execution of a program. Such a view leads to a notion of program life that depends on the continued support of the program by programmers having its theory. Further, on this view the notion of a programming method, understood as a set of rules of procedure to be followed by the programmer, is based on invalid assumptions and so has to be rejected. As further consequences of the view, programmers have to be accorded the status of responsible, permanent developers and managers of the activity of which the computer is a part, and their education has to emphasize the exercise of theory building, side by side with the acquisition of knowledge of data processing and notations.

3.2 The psychopathology of everyday things

- **Principles of how people interact with things:**

- **Visibility:** Es uno de los más importantes principios de diseño, las partes correctas de la cosa deben ser visibles, y debe ser claramente perceptible y comprensible el mensaje correcto de lo que se puede hacer.
- **mapping:** La relación entre los controles del objeto y sus efectos debe ser lógico y comprensible.
- **Feedback:** Las cosas deben comunicar con algún comportamiento para saber si lo que se hizo tuvo efecto y que efecto fue el que tuvo.

- **Natural design:** El uso de señales naturales, son señales naturalmente interpretadas sin necesidad de ser consciente de ellas.

- **Constraints:** Es la propiedad de un objeto que sugiere como NO debe ser usado. Un diseño sugiere de manera intuitiva como debe interactuar uno con él. Si estas son claras, el uno sabe intuitivamente que debe hacer.

- **Affordances:** Es la propiedad de un objeto que sugiere como debe ser usado. Un diseño sugiere de manera intuitiva como debe interactuar uno con él. Si estas son claras, el uno sabe intuitivamente que debe hacer.

- **Conceptual model:** Representación mental que el usuario crea acerca de como funciona un objeto basándose en su experiencia previa y en las pistas que ofrece el diseño. El modelo mental del usuario se debería alinear con el funcionamiento real del objeto.

- **Simplicity:** Crear un equilibrio entre las capacidades del objeto y la facilidad de uso. Eliminar funciones o detalles innecesarios para que el objeto sea fácil de usar. La paradoja de la tecnología nunca debería ser usada como excusa para hacer un diseño pobre.

3.3 No Silver Bullet (by Frederic P. and Brooks Jr.)

- **Abstract:**

- La construcción de software implica:
 - * **Essential tasks:** Modelamiento (fashioning) de las complejas estructuras conceptuales que componen la entidad abstracta del software.
 - * **Accidental tasks:** La representación de estas entidades abstractas en lenguajes de programación y la relación de estos en los lenguajes de la máquina en un tiempo y velocidad determinados.

- **Does it have to be hard? - Essential difficulties**

- Para ver que rate de progreso puede tener el software, hay que ver las dificultades
 - * Dificultades de la esencia (que son inherentes en la naturaleza del software)
 - * Dificultades accidentales (que hoy aparecen en la producción pero no son inherentes)
- **Complexity**
 - * Son más complejas por su tamaño ya que no son dos partes iguales (en general).
 - * Aumenta la complejidad el hecho de que los elementos del software interactúen entre ellos, lo cual lo hacen de una forma no lineal.
 - * De la complejidad sale:
 - Complejidad en enumerar/entender todos los posibles estados del programa
 - Complejidad en las funciones (lo cual genera dificultades invocando estas, lo cual hace el programa difícil de usar).
 - Complejidad de estructura: Que dificulta el extender el programa con otras funciones sin tener efectos secundarios; y también genera estados sin ver que hacer puertas que afectan la seguridad.
- **Conformity**
 - * Mucha de la complejidad viene de la conformación a otras interfaces, lo cual no puede ser simplificado por un rediseño de software solamente.
 - * Las interfaces difieren entre sí ya que fueron diseñadas por distintas personas en vez de por dios.
- **Changeability**
 - * La entidad de software siempre está siendo apurada a cambiar.
 - * El producto de software está embebido en una matriz cultural de aplicaciones, usuarios, leyes, y vehículos de máquinas que cambian constantemente y estos cambios forzan al cambio en el producto del software.
- **Invisibility**
 - * El software es invisible ya que la realidad del software no está inherentemente incorporado en el espacio.
 - * Si se quiere intentar hacer un gráfico del software seguramente se pierda información conceptual e impida un mejor desarrollo.
 - * No solo impide el proceso de diseño con el proceso de la misma mente de uno, si no que dificulta la comunicación entre mentes también.

- **Past Breakthroughs Solved Accidental Difficulties**

- Los 3 más grandes pasos en el desarrollo de la tecnología de software fueron hechos atacando dificultades accidentales, también estos tienen límites para la extrapolación de los mismos. Estos son:
 - * **High-level languages:** Ataca la productividad de software, confiabilidad, y la simplicidad.
 - * **Time-sharing:** Ataca la productividad de los programadores y la calidad de sus productos, preserva la inmediatez y por ende permite mantener una visión general de la complejidad.
 - * **Unified programming languages:** Ataca la dificultad accidental de usar programas juntos.

- **Hopes for the silver**

- **Ada and other high-level language advances**
- **Object-oriented programming**

- Artificial intelligence
- Expert systems
- "Automatic" programming
- Graphical programming
- Program verification
- Enviroments and tools
- Workstations
- Promising Attacks on the Conceptual Essence
 - Buy versus build
 - Requirements refinement and rapid prototyping
 - Incremental development grow, not build, software
 - Great designers

3.4 SELF: The Power of Simplicity

- Introduction
 - La programacion orientada a objetos da una perspectiva util para disenar programas de la computadora.
 - **SELF:**
 - * No tiene clases ni variables.
 - * No hay diferencia entre acceder a una variable y mandarle un mensaje.
 - * No tiene estructuras de control, closures y polimorfismo apoyan/sustentan (support) estructuras de control arbitrarias en el lenguaje.
 - * Los objetos y procedimientos son representados por prototipos de memoria perdida para la invocaion de estos.
 - * Ademas de que comparten el mismo modelo de creacion (clonando prototipos), los procedimiento salmanan sus variables y mantienen su informacion de ambiente de la misma manera que objetos ordinarios.
- Compariong Prototypes and Classes
 - **Simpler Relationships:** En vez de tener que hacer la relacion "es un" (el objeto es una instancia de clase) y "del tipo" (si la clase es una subclase de otra clase de otro objeto), SELF tiene la relacion "hereda de" (describe como los objetos comparten comportamiento y estado), esto hace dificil la distincion entre cuales objetos existen solamente para proveer informacion compartida para otros objetos.
 - **Creation by copying:** A diferencia de Smalltalk (el cual usa instanciacion para la creacion de objetos) SELF usa la copia de prototipos para cerar nuevos objetos.
 - **Examples of preexisting modules:** Prototypes son ejemplos de objetos (mas que descripciones de formato e inicializacion como en Smalltalk). Un sistema basado en prototipos permite examinar un representante tipico mas que requerir entender la descripcion de este.
 - **Support for one-of-a-kind objects:** Cualquier objeto puede tener slots unicos o comportamiento unico a diferencia de un sistema basado en clases.
 - **Elimination of meta-regress:** Ningun objeto en un sistema basado en clases puede ser auto-suficiente. Si el sistema basado en prototipos no tiene la posibilidad de heredar entonces no tiene la posibilidad de hacer cambios rapidos cambiando comportamiento compartido.
- Blending State and Behavior
 - No hay una forma directa para acceder a una variable en SELF, en cambio los objetos envian mensajes para acceder a la informacion en los slots con nombres.
 - Acceder a estados por mensajes hace que la herencia sea mas poderosa.
- Closures and Methods

- **Local variables:** Los objetos que juegan el papel de metodos son prototypes de activation records(region de la memoria pedida durante una llamada de un procedimiento para alocar diversas cosas para la invocacion del procedimiento); estos soon copiados para correr la subrutina. Entonces, las variables locales son alocadas reservando slots para estas en el prototype activation record.
- **Environment link:** Si un slot no se encuentra en el scope actual, mira al siguiente scope de afuera mirando el parent link.

- **Conclusions**

- La eliminacion de clases y variables hace desaparecer el metaclass regress, difumina la ilusion de distincion entre instanciacion y subclasear(? (subclassing).
- Como la variedad de construccion cesa, tambien lo hace la variedad de pistas linguisticas a la estructura del sistema.

3.5 Design Principles behind Smalltalk (hecho por chatGPT)

- **Personal Mastery**

- El primer principio subraya que las computadoras deben estar al servicio de las personas y facilitar su uso. La idea es que el sistema debería empoderar a los individuos, permitiéndoles aprender y controlar la máquina con facilidad.

- **Simplicidad**

- Un diseño simple es clave. Esto se traduce en crear un sistema que pueda ser explicado de manera concisa. En Smalltalk, cada elemento debe ser fácil de entender y explicar, manteniendo un nivel bajo de complejidad técnica.

- **Uniformidad**

- La uniformidad en el diseño es esencial para asegurar que cada parte del sistema funcione bajo un mismo conjunto de reglas o conceptos. En Smalltalk, todo es un objeto, desde datos básicos hasta estructuras complejas, lo que genera una mayor coherencia en su uso y aprendizaje.

- **Potencia**

- Smalltalk debe ser suficientemente potente para resolver cualquier problema de programación. Aunque se aprecie la simplicidad, el lenguaje no debe ser tan limitado que se vuelva inútil para problemas complejos. El sistema tiene que proporcionar herramientas potentes para construir cosas complejas con facilidad.

- **Incrementalismo**

- El sistema debe permitir mejoras graduales y cambios dinámicos. Smalltalk está diseñado para ser flexible y adaptable, facilitando la creación y modificación del software sin requerir que el sistema se detenga. Este principio apunta a la idea de que cualquier cambio en el sistema debe ser rápido y sin interrupciones.

- **Reflexividad**

- Smalltalk es un sistema reflexivo, lo que significa que es capaz de observar y modificar su propia estructura y comportamiento en tiempo de ejecución. Esto hace que sea fácil de depurar y extender, ya que el sistema en sí está construido con los mismos objetos y principios que utilizan los programadores para desarrollar aplicaciones.

- **Apertura**

- El sistema debe ser abierto, lo que implica que los usuarios puedan inspeccionar, comprender y modificar cualquier parte de este. La idea es que todo, desde las primitivas más básicas hasta las interfaces de usuario, debería estar accesible para su exploración y mejora.

- **Extensibilidad**

- El lenguaje está diseñado para que pueda extenderse de manera indefinida. En Smalltalk, las primitivas del lenguaje no son sagradas, sino que se pueden redefinir o extender según sea necesario.

- **Conclusion**

- Smalltalk fue diseñado para ser un sistema que empodera al usuario, es simple de entender y consistente en su enfoque de "todo es un objeto". Al mismo tiempo, busca mantener la potencia suficiente para resolver problemas complejos, mientras permite el cambio continuo y la evolución sin interrupciones.

3.6 Polymorphic Hierarchy

- **Conclusions**

- Para que dos metodos sean polimorficos, estos no solo tienen que tener el mismo nombre, si no tambien el mismo tipo de parametros, los mismos efectos secundarios y el mismo tipo de retorno.
- Dos metodos que son polimorficos deberian aparecer en el mismo method protocol.
- Una descripcion de metodo debe tener dos cosas:
 - * **Purpose details:** Explica que hace el metodo.
 - * **Implementation details:** Es opcional, serian como aclaraciones o justificaciones de por que el codigo es como es.
- Para que dos metodos sean polimorficos, deben tener el mismo purpose, pero no los mismos implementation details.
- Para que dos clases sean polimorficas deben tener la misma core interface (subconjunto de metodos que varias clases comparten para que se puedan usar intercambiabilmente) de mensajes polimorficos.
- Un colaborador puede usar las dos clases intercambiabilmente si solo usa mensajes en la core interface.
- Para que una jerarquia sea polimorfica, todos sus clases deben compartir la misma polymorphic core interface.
- La jerarquia polimorfica y su core interface son definidas por una clase abstracta (cumple el rol de una Template Class).
- Una jerarquia polimorfica encapsula codigo que es altamente reutilizable, flexible y extensible.

3.7 A Simple Technique for Handling Multiple Polymorphism

- **Abstract**

- Las expresiones que tienen varios terminos con cada una teniendo un variable type sugiere que se debe recurrir a multiple polymorphic expressions.

- **The Problem**

- Cuando mas de una variable en una expresion es independientemente polimorfica lleva a un estado de codear que jode con la explicita forma de testing.

- **The Solution**

- Es necesario entender la coneccion entre el polimorfismo y el mensaje enviandose para reconocer el acercamiento adecuado.
En esencia, cada mensaje transmitido reduce a una variable polimorfica a una monomorfica con el tipo de dispatch inherente en el message lookup.
Usualmente (por diseno), solo el receptor es polimorfica y la situacion es simple. Aunque en el caso de que no, el primer dispatch hace una parte del trabajo solamente y se debe enviar otro mensaje para reducir el polimorfismo restante.
- Cada subsequent message dispatch reduce un grado de polimorfismo.

3.8 The Null Object Pattern

- **Intent**

- The Null Object encapsula la implementacion de decisiones de como "hacer nada".

- **Motivation**

- La clase que desea tratar a un colaborador que no hace nada es la misma que la que da behavior.
- The Null Object Pattern describe como desarrollar una clase que encapsula como un tipo de objeto deberia hacer nada. Como el codigo de "do nothing" esta encapsulado, su complejidad esta oculta de sus colaboradores y puede ser facilmente reutilizada por cualquier colaborador que lo quiera.
- La clave del Null Object pattern es una clase abstracta que define la interfaz para todos los objetos de este tipo. The Null Object es implementado como una subclase de su clase abstracta. Como es parte de la interfaz de la clase abstracta, puede ser usado en cualquier lugar que este tipo de objeto necesite.

- **Applicability**

- Se deberia usar el Null Object pattern cuando:

- Un objeto precisa un colaborador. The Null Object pattern no introduce esta colaboracion, usa una colaboracion que ya existe.
- Algunas instancias de colaboradores deberian no hacer nada.
- Queres que tus clientes puedan ignorar la diferencia entre un colaborador con comportamiento y otro que no haga nada. De esta forma, el cliente no tiene que fijarse manualmente por un nil u otro valor especial.
- Queres que se pueda reusar el comportamiento de "do nothing" para que para varios clientes que necesiten este comportamiento obtengan el mismo comportamiento.
- Todo el comportamiento que puede necesitar ser "do nothing" es encapsulado en la clase del colaborador. Si algunos de los comportamientos en la clase es un comportamiento de "do nothing", la mayoría o todo el comportamiento de la clase seria "do nothing".

- **Collaborations**

- Si el receiver es un RealObject, entonces el pedido es manejado para dar un real comportamiento. Si el receiver es un NullObject, el pedido es manejado por no hacer nada o a lo sumo dando un resultado null.

- **Consequences**

- The Null Object pattern:
 - * define class hierarchies consistiendo en objetos reales y objetos null.
 - * hace el codigo para el cliente simple.
 - * encapsula el codigo de "do nothing" en objetos null.
 - * hace que el codigo de "do nothing" en el objeto null sea facil de reutilizar.
 - * hace que el comportamiento de "do nothing" sea dificil de distribuir o mezclar en el comportamiento real de varios objetos colaboradores.
 - * puede necesitar crear una nueva clase NullObject por cada nueva clase de AbstractObject.
 - * siempre actua como un objeto que no hace nada. No se puede transformar en un Real Object.

3.9 Method Object

- **Resumen**

- Para resolver el problema de que un metodo tenga muchas lines de codigo en las que se comparten muchos argumentos y variables temporales se puede usar este metodo.
- La solucion es crear un objeto que represente una invocacion del metodo y use los nombres de las variables de instancia en el objeto para habilitar futura simplificacion.
- Muchos objetos son sustantivos, estos son verbos.
- No son faciles de explicar al no tener nada analogo en el mundo real.
- Para ver el ejemplo ir al paper.

4 Papers del segundo parcial

4.1 The Object Recursion Pattern - Booby Woolf

- **Intent**

- Distribuye el procesamiento de un pedido sobre una estructura delegando polimórficamente.
- Permite el pedido de ser separada en partes mas pequenas que son mas sencillas de manejar.

- **Motivation**

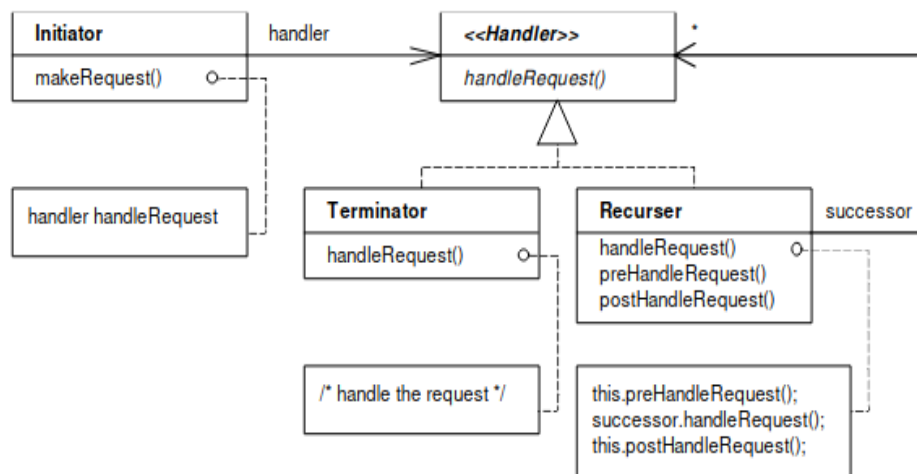
- Un ejemplo es si uno quiere implementar la comparacion de se 2 objetos y lo quiere hacer con otro objeto que lo haga, este tendria que depender directamente de los 2 objetos a comparar (O sea, si cambia la clase polimorfica de la que estos dependen tambien se veria afectado el objeto comparador), ya que el mismo tendria que cambiar cada vez que la implementacion cada vez que cambie la implementacion de la clase de los objetos comparados, lo cual lo vuelve muy dificil de mantener y desarrollar.
- El comparador tendria que decirle a los objetos **que hacer** y no **como hacerlo** para resolver lo anterior.
- Uno de los objetos a comparar se tiene que fijar cuales partes del otro objeto deberian ser equivalentes y compara estas, asi hasta llegar a un objeto primitivo donde la comparacion ya esta resuelta y no es recursiva.
- Se compara a si mismo con el otro objeto diciendole a sus partes que se comparen entre ellas y asi sucesivamente, es un mensaje recursivo que envia el mismo mensaje a uno o mas objetos relacionados por varias iteraciones hasta llegar al metodo terminal del mensaje.

- **Keys**

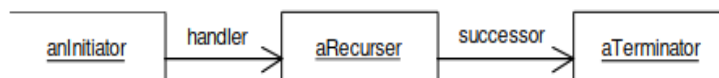
- Un sistema que incorpora Object Recusion pattern tiene las siguientes características:
 - * Dos clases polimorficas, una que puede handlear el pedido recursivamente y otra que simplemente la handlea sin la recursion.
 - * Un mensaje separado, que esta generalmente en una tercera clase que no es polimorfica con las otras dos para inicializar el pedido.

Structure

The following class diagram shows the roles of the participants:



A typical object structure might look like this:



- **Applicability**

- Se puede usar el Object Recursion pattern cuando:
 - * Se quiere pasar un mensaje por una estructura linkeada donde no se conoce el destino final.
 - * Se quiere comunicar un mensaje a todos los nodos de una estructura linkeada.
 - * Se quiere distriibuir una responsabilidad de comportamiento en toda una estructura linkeada.

- **Consequences**

- Las ventajas de Object Recursion pattern son:
 - * **Distributed processing:** El procesamiento de un pedido es distribuido a lo largo de una estructura de Handlers que puede ser tan numerosa y compleja como sea necesario para completar la tarea.
 - * **Responsibility flexibility:** El Initiator no necesita saber cuantos Handlers hay o como estan acomodados, solamente hace el pedido a su Handler y deja que los handlers se ocupen.
 - * **Role flexibility:** Un Handler que funciona como un Recurser de uno de los pedidos puede actuar como Terminator para otro pedido y viceversa.
 - * **Increased encapsulation:** Encapsula las decisiones de como manejar un pedido dentro del objeto que lo esta manejando.
- Las desventajas de Object Recursion pattern son:
 - * **Programming complexity:** El uso excesivo de este patron puede hacer a un sistema mas dificil de entender y mantener

4.2 Pattern Abuser - Mahesh Dodani

- **Resume**

- El paper comienza con Pat diciendo que antes su tecnica de dieño era comenzar con algo cercano a lo que queria y luego modificarlo hasta que funcione.
- Luego, JERR le muestra como el codigo que habia hecho se podria refactorizar con patrones y como quedaba mejor (mas facil de entender, de modificar, testear y mantener para la gente que enttendia el codigo secreto de los patrones definidos en el libro).
- Pat se obsesiona con los patrones de diseño y comienza a aplicarlos en todos lados y evangelizar a sus colegas con estos, llego al punto de medir que tan bueno era un codigo en terminos de cuantos patrones habian sido utilizados, esta medida fue usada para medir reusabilidad.
- Se vuelve una celebridad de los patrones y aplicaba patrones sin discriminacion cuando sea posible, sin importarle las consecuencias que tiene en el codigo.
- Llega al punto de desarrollar un patron para aplicar patrones.

- **"Pattern" for applying patterns**

- **For the instance variables, apply the State Pattern**
- **For each method, aplly the Strategy Pattern**
- **For grouping of instances, apply the Chain of Responsibility Pattern**
- **For a grouping of instances, apply the Composite, Iterator, and Visitor Patterns**
- **For clients of objects, apply the Observer Pattern**
- **For clients of objects, apply the Proxy Pattern**
- **For each hierarchy of objects, apply the Facade Pattern**
- **For complex methods, apply the Bridge Pattern**
- **For "legacy" objects, apply the Adapter Pattern**
- **For creating objects as part of a family, apply the Abstract Factory, Builder, and Factory Method Patterns**
- **For creating similar objects, apply the Prototype Pattern**

- **The dark side**

- Reflexionando en los sistemas que habia construido se puso a pensar que no sabia si los sistemas estaban mejores o peores luego de aplicarle el patron de patrones.
- Cuenta que una vez cuando la llamaron para refactorizar el codigo de un banco, cuando se fue la tuvieron que llamar de nuevo porque los desarrolladores del banco no podian hacer cambios facilmente en el sistema.
- Se desligo de esta situacion diciendo que los problemas que tenian eran que:
 - * Los desarrolladores del banco necesitaban tiempo para madurar en la forma de pensar y usar patrones
 - * El problema que ellos identificaron podria ser resuelto introduciendo mas patrones en al aplicacion.
- Su discurso los convencio a los del banco y la contrataron de nuevo para hacer los cambios necesarios y entrenar a sus desarrolladores
- Comienza a reflexionar y se plantea:
 - * Como se que un patron es realmente aplicable y va a tener el impacto intencionado en la aplicacion?
 - * Como se si la aplicacion el patron del patron hizo a esta mas reusable?
 - * Va a ser la aplicacion mejor luego de aplicarle el patron?
 - * Como defino mejor y determino que la aplicacion sea mejor?
- Can you please help me?

4.3 Respuesta sobre Compute Science de Alan Kay en Quora

- **Pregunta**

- Was computing dumbed down by the arrival of computer science in academia?

- **Respuesta**

- En un inicio, pensaba que no, luego penso que al rededor del año 1980 cuando la computacion en la academia se hizo mas conocida se empezo a dar en muchas universidades cuando no habia posiblemente la cantidad de profesores calificados para enseñar.
- Otro factor es que las universidades se orientaron mas hacia la formacion vocacional y los negocios, en lugar de un entendimiento profundo de problemas complejos.
- Aclara que no sabe lo suficiente para responder esta pregunta directamente pero que por lo que ha recolectado en visitas y charlas a departamentos universitarios es que los estudiantes no saben sobre la historia de la computacion y especialmente la ciencia de la computacion.
- Dice que "La computacion no esta siendo enseñada como un campo laboral real" (por ejemplo en fisica no ves solamente $F = ma$, si no que ves a quien se le ocurrio).
- Estos comportamientos por parte de las universidades y los alumnos indicaba una real falta de perspectiva y curiosidad intelectual.
- Habla tambien de que no encontro a ningun estudiante que pudiese definir "computer science" como algo que no sea simil ingenieria. El termino ciencia no tenia ninguna coneccion con su significado del siglo XVII.

- **Respuesta final**

- Esta claro que "computer science" fue desastrozamente atontada. Cree que la academia puede ser culpada por la mayor parte del problema (porque parte del trabajo es "mantener la llama" protegidas del atontamiento general en el mundo). Cree que los negocios pueden ser culpados por "ser tontos": Afectan a un nivel general de la poblacion de varias maneras, incluyendo la enseñanza en todos sus niveles.

4.4 Modern Software Engineering - David Farley

- **(Cap 1) Engineering - The practical Application of Science**

- Los software engineers necesitan convertirse en expertos en aprender.
- Tendriamos que aplicar tecnicas y estrategias de la ciencia, haciendo referencia a que tendriamos que aplicar algunas de las basicas, pero extremadamente importantes ideas.

- Por ejemplo el metodo cientifico que consiste en:
 - * **Characterize:** Hacer una observacion sobre el estado actual.
 - * **Hypothesize:** Crear una descripcion, una teoria que pueda llegar a explicar la observacion.
 - * **Predict:** Hacer una prediccion basada em la hipotesis.
 - * **Experiment:** Testea la prediccion.
- Cuando organizamos el pensamiento de esta manera y empezamos a progresar desde lo basico en muchos experimentos informales y pequeños, se empieza a limitar el riesgo de terminar teniendo conclusiones inapropiadas y se termina haciendo un mejor trabajo.
- Si se comienza a pensar en funcion de controlar las variables en el experimento se llega a un sistema y codigo mas deterministico.
- En cambio, si se piensa en terminos de ser esceptico sobre las ideas que tenemos y explorar como podriamos falsificarlas, podemos identificar, y entonces eliminar ideas malas en seguida y hacer el progreso mas rapido.
- Lo que se busca en este libro es adoptar los principios cientificos basicos, en otras palabras, **engineering**.

● What is Software engineering?

- **Software engineering** es la aplicacion de un enfoque empirico y cientifico en encontrar soluciones eficientes y economicas a problemas en el software.
- El enfoque de engineering tiene dos puntos importantes:
 - * El desarrollo de software es siempre un ejercicio de descubrimiento y aprendizaje.
 - * El objetivo es ser "eficiente" y "economico", entonces nuestra habilidad de aprender debe ser sustentable.
- El objetivo es ser **expertos en aprender y manejar complejidad**
- Para ser **expertos en aprender** necesitamos las siguiente tecnicas:
 - * Iteration
 - * Feedback
 - * Incrementalism
 - * Experimentation
 - * Empiricism
- Necesitamos poder ser capaces de trabajar en formas que nos faciliten progresar cuando las respuestas, y algunas veces incluso la direccion es incierta.
- Para ser **expertos en manejar complejidad** necesitamos:
 - * Modularity
 - * Cohesion
 - * Separation of Concerns
 - * Abstraction
 - * Loose Coupling
- Algunas herramientas practicas para llevar una estrategia efectiva para cualquier desarrollo de software son:
 - * Testability
 - * Deployability
 - * Speed
 - * Controlling the variables
 - * Continuous delivery
- Con este pensamiento se crea software de mejor calidad, mas rapido y la gente trabajando en equipos que adoptan estos principios han dicho que disfrutan mas su trabajo.

● Reclaiming "Software Engineering"

- En otras disciplinas, **engineering** es el proceso y practica que aplicas para incrementar tus probabilidades de hacer un buen trabajo.

- En cambio, en nuestra disciplina se refiere a apuntar a describir un modelo intelectualmente consistente que pone a trabajar en conjunto principios fundacionales que estan en las raices de todos los buenos desarrollos de software.
- Nunca hay garantia de exito, pero estas herramientas y principios aumentan tus chances de exito.

- **How to make progress**

- Para permitirnos la libertad intelectual de desafarnos y refutar los dogmas y diferenciar entre moda, ideas malas y buenas ideas, cualquier sea su fuente necesitamos una estructura que nos permita crecer y involucrarnos con con enfoques, estrategias, procesos, tecnologias y soluciones mejorados, a esta estructura la llamamos **ciencia**.
- Cuando aplicamos este tipo de pensamiento en resolver problemas practicos lo llamamos **engineering**.

- **The birth of software engineering**

- Fue un concepto creado en los 60's por Margaret Hamilton.
- Una observacion interesante es que el desarrollo de hardware es tan rapido que el desarrollo de software le quedo corto en rapidez no porque sea lento si no porque el otro es muy rapido.

- **Shifting the Paradigm**

- La idea de cambio de paradigma fue creada por el fisico Thomas Kuhn.
- Un tipo de aprendizaje es el que se forma de capas de entendimiento, con cada capa con cada capa fundamentalmente conectada con la anterior.
- Pero tambien hay otro en el que se cambia fundamentalmente nuestra perspectiva sobre algo, esto nos permite aprender cosas nuevas, pero tambien significa que debemos descartar lo que estaba antes.
- La idea de cambio de paradigma implicitamente incluye la idea de que cuando hacemos tal cambio, como parte del proceso, eliminamos ideas que sabemos ahora que no son mas correctas.
- Tratar el desarrollo de software como una ingenieria genuina implica fundamentarlo en la filosofia del metodo cientifico y el racionalismo cientifico.
- Esto nos permite un enfoque de aprendizaje mas efectivo y facilita el descarte ideas eficientemente.

- **Summary**

- Aplicar este tipo de pensamiento ingenieril al software no necesita ser pesado o extremadamente complejo. El cambio de paradigma en pensar diferente sobre lo que se hace y como se hace cuando creamos software deberia ayudarnos a verlo mas sencillamente, mas confiable y eficiente.
- Esto se trata de mejorar nuestra habilidad para crear software de alta calidad mas sostenible y confiable.

- **(Cap 2) What is engineering?**

- Hacer un puente no es hacer software.
- Production engineering es un problema complejo cuando la disciplina involucrada tiene que manejar cosas fisicas, estas se necesitan con un nivel de precision y calidad.

- **Production is not our problem**

- En la industria del software se ha intentado constantemente aplicar el "pensamiento/estilo de produccion", pero esto generalmente usa herramientas que no son las de descubrimiento, aprendizaje y experimentacion que necesita el software.
- Production is not our problem.

- **Design Engineering, not production engineering**

- Los ingenieros en otras disciplinas adoptan tecnicas de modelamiento, simulacion en computadora las cuales luego las tienen que probar fisicamente, este no va a ser acertado. En cambio las simulaciones de los modelos que se crean en software son el producto final; nuestros modelos son la realidad de nuestro sistema.
- No tenemos que preocuparnos, entre otras cosas en el costo de cambiar nuestro sistema, estos son dramaticamente menos costosos que los de cambiar un puente.

- Nuestra disciplina es tecnica.
- Un error muy cometido es asumir un nivel idealistico de precision que es imposible en cualquier campo, incluyendo al desarrollo de software (ya que se confunde que como el desarrollo de software tendria que ser practicado con un razonamiento cientifico se debe buscar precision matematica).

- **Engineering as Math**

- * Un metodo formal es una forma de construir sistemas de software, que implica una validacion matematica del codigo escrito para asi probar que el codigo es correcto.
- * Un metodo formal para construir software solo funciona para casos muy acotados, donde el problema es lo suficientemente sencillo, delimitado, deterministico y con pocas variables. Si el sistema es un poco mas complicado la prueba con este metodo se vuelve imposible.
- * Por esto, tomamos el mismo camino que los de ingenierias aeroespaciales por ejemplo, aplicamos pensamiento matematico hasta donde podamos, despues tomamos un camino guiado por datos, pragmatico, empirico y experimental para el aprendizaje, permitiendonos adaptar nuestros sistemas mientras que los hacemos crecer incrementalmente.
- Como los modelos que creamos en software son el resultado ejecutable de nuestro trabajo, nosotros testeamos nuestros productos, no nuestro mejor acercamiento a la realidad de nuestros productos.
- La simulacion experimental del "mundo real" para testear nuestros productos es mucho mas acertada que en cualquier otra disciplina.
- Como el desarrollo de software se basa en descubrimiento, aprendizaje y diseño se tendrian que aplicar tecnicas de exploracion mas que tecnicas de produccion ingenieriles.

- **The first software engineer**

- * Una de las fuerzas detras de el enfoque de Hamilton era el enfocarse como las cosas fallan (en que formas llegamos a hacer que las cosas fallen).
- * La asuncion no era que podias planear y llegar a hacer que las cosas funcionen bien la primera vez, si no que tratas a todas las ideas, soluciones, y diseños con escepticismo hasta que te quedes sin ideas de como las cosas podrian salir mal. Esto es el ejemplo de como funciona la **empirical engineering**.
- * El otro principio de ingenieria que fue parte de el trabajo de Hamilton fue la idea de "failing safely". La asuncion es que no puedes codear para cada escenario posible, entonces lo que se hace es que se codea un error sin ninguna prediccion especifica de cuando o como puede llegar a ser util para que no se rompa todo por un error que no fue considerado.
- * Entonces dos atributos fundamentales para Hamilton fueron empirical learning y el descubrimiento y habito de imaginar como las cosas podrian a llegar a funcionar mal.

- **A working definition of engineering**

- Engineering es la aplicacion de un enfoque empirico y cientifico de encontrar soluciones eficientes y economicas a problemas practicos.
- Engineering es una ciencia aplicada, por lo que es practico.
- Empirico significa que se aprende y avanza entendiendo las soluciones con respecto a la solucion al problema.
- Las soluciones son practicas y aplicables al problema y el contexto.
- Son eficientes y estan creadas con un entendimiento de y restringidas por las economias de la situacion.

- **(Cap 3) Fundamentals of an engineering approach**

- Engineering en diversas disciplinas varia su significado pero todas coinciden en que estan firmemente fundamentadas en racionalizamiento cientifico y toman un enfoque pragmatico y empirico en hacer progreso.
- Si se quisiera hacer un conjunto de ideas, practicas, y comportamientos para definir software engineering, estas ideas van a ser acordes a la realidad de desarrollo de software pero robustas en la cara del cambio.

- **An industry of change?**

- Mucho del cambio en nuestra industria es efimero y no mejora las cosas, algunos incluso las empeora.

- La velocidad del progreso del software es mucho mas lento de lo que se considera. Esto lo puedes ver preguntandote que cambios en tu carrera tuvieron un impacto en la forma en la que piensas y aplicas desarrollo de software? La lista es mas corta de lo que se asume.
- Los pasos de assembler a C y de procedural a POO fueron pasos que representaron cambios significativos en el nivel de abstraccion que podia manejar al escribir codigo, cada uno representaba un cambio en la complejidad de los sistemas que podiamos construir. Los lenguajes por si mismos son menos importantes que el paradigma de programacion.
- No solo nos parece dificil encontrar nuevas ideas si no que parece ser casi imposible el descartar ideas viejas.

• The importance of Measurement

- Uno de los motivos por los que se nos hace dificil descartar malas ideas es que no tenemos una forma de medir el desempeño en el desarrollo de software eficientemente.
- La mayoría de las metricas aplicadas en medir el desarrollo de software son o irrelevantes (velocidad) o a veces dañinas (lineas de codigo o abarcamiento de los tests).
- No tenemos una medida para productividad.
- Una forma presentada por Nicole Fosgren, Jez Humble, y Gene Kim es que se podria evaluar la eficiencia del desarrollo de software basandose en 2 atributos clave, **estabilidad y rendimiento (throughput)**. No pueden probar que estas medidas tienen una relacion causal con el desempeño de los equipos de desarrollo, pero pueden demostrar una correlacion estadistica.
- La estabilidad se mide por lo siguiente:
 - * **Change Failure Rate:** La tasa en la que un cambio introduce un defecto en un momento particular del proceso.
 - * **Recovery Failure Time:** Caunto tiempo se tarda en recuperarse de un fallo en un punto particular del proceso.
- El rendimiento se mide por lo siguiente:
 - * **Lead Time:** Una medida de la eficiencia en el proceso de desarrollo. Cuanto se tarda en un solo cambio pasar de una idea a software funcionando?.
 - * **Frequency:** Una medida de velocidad. Que tan a menudo hay cambios mandados a produccion?.
- Estas medidas tecnicas del enfoque de desarrollo responden a las preguntas "Cual es la calidad de nuestro trabajo?" y "Que tan eficiente podemos crear trabajo de esta calidad?".
- Esta forma deja algunos aspectos muy importantes sin considerar, no ve si se esta construyendo lo correcto si no que las estemos construyendo bien.

• Applying Stability and Throughput

- El model of stability and throughput ofrece la oportunidad de usarlo para evaluar cambios en nuestro proceso, organizacion, cultura o tecnologia. Sirve para hacer decisiones basandonos en evidencia mas que adivinando.
- **Al fin tenemos una vara de medicion que sirve para algo**
- La idea seria que cada vez que se quiera hacer un cambio se mida con este modelo y ver si cambia el valor de las mediciones del modelo.
- Hay que tener en cuenta el significado del resultado de estas mediciones, pero tener mediciones que por lo menos significan algo importante nos permite evaluar las acciones que se toman, lo cual llega a ser hasta vital incluso para tomar un enfoque basado mas en evidencias a la hora de tomar decisiones.

• The Foundations of a Software Engineering Discipline

- Cuales son las ideas que podriamos llegar a esperar que sigan siendo correctas en 100 años?
- Nuestra disciplina se tiene que enfocar en 2 aspectos esenciales, el que nos deberiamos convertir expertos en aprender y expertos en manejar complejidad (tanto en un nivel tecnico como organizacional).

• Experts at Learning

- Habla de lo mismo que puso en la introduccion, el desarrollo de software es un ejercicio de exploracion y descubrimiento donde uno aprende a organizarse mejor y entiende mas profundamente los problemas en los que se trabaja.

- **Experts at Managing Complexity**

- Habla tambien de lo mismo que en la introduccion donde menciona las 5 ideas ara manejar complejidad de una manera estructurada para cualquier sistema de informacion.

- **Summary**

- Las ideas que facilitan nuestro aprendizaje y nos permiten manejar la complejidad de los sistemas que creamos son las herramientas reales que tenemos.
- Tener algo para medir nuestro desarrollo de software es una gran ventaja si se quiere tomar decisiones basandose en evidencia y datos mas que en moda o adivinar.
- Para hacer una decision de cambio nos tenemos que preguntar "Aumenta la calidad del software que creamos?" (stability) o "Aumenta la eficiencia con la cual creamos el software de esta calidad?" (throughput).

4.5 Design Patterns

- **Composite**

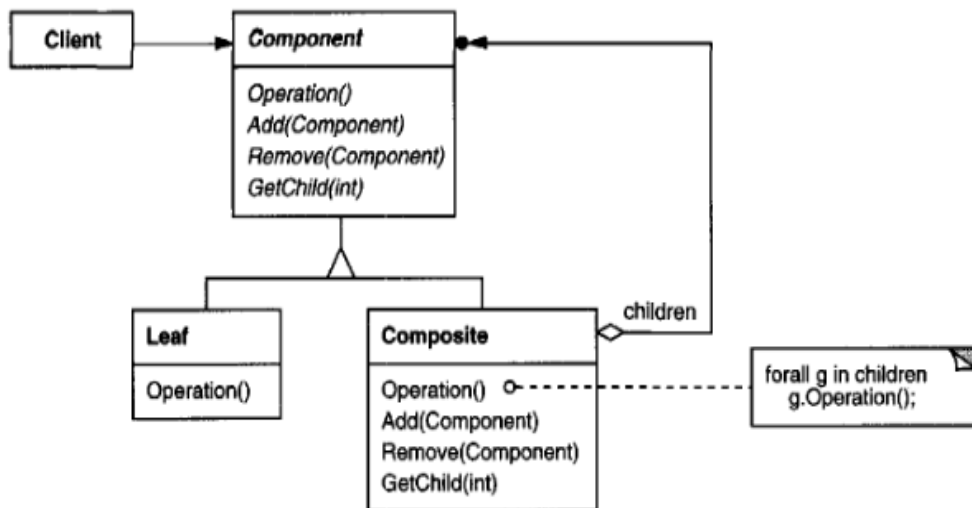
- **Intent**

- * Componer objetos en estructuras de arboles para representar una parte/toda una jerarquia. Composite permite a los clientes a tratar objetos individuales y composiciones uniformemente.

- **Applicability**

- * Usar Composite cuando:
 - Queres representar una parte/toda una jerarquia de objetos.
 - Queres que los clientes sean capaces de ignorar la diferencia entre composiciones de objetos y objetos individuales. Los clientes van a tratar todos los objetos de la estructura composite uniformemente.

Structure



– Participants

* Component

- Declara la interfaz para los objetos en la composicion
- Implementa el comportamiento predeterminado de la interfaz comun a todas las clases, como corresponde.
- Declara una interfaz para acceder y manejar sus componentes hijos.
- (Opcional) Define una interfaz para acceder al padre de un componente en la estructura recursiva, y lo implementa si corresponde.

* Leaf

- Representa una hoja en la composicion. No tiene hijos.
- Define el comportamiento de objetos primitivos en la composicion.

* Composite

- Define el comportamiento de componentes con hijos.
- Almacena a los componentes hijos.
- Implementa operaciones relacionadas con los hijos en la interfaz del componente.

* Client

- Manipula los objetos en la composicion mediante la interfaz del componente

– Collaborations

- * Los clientes usan la interfaz Component class para interactuar con los objetos en la estructura composite. Si el recipiente es un Leaf, entonces el pedido se maneja directamente. Si el recipiente es un Composite, entonces generalmente se envia el pedido a los hijos del componente, posiblemente haciendo mas operaciones antes y/o despues de enviarlo.

– Consequences

- * Define jerarquias de clase consistiendo en objetos primitivos y compuestos. Los primitivos pueden tambien se pueden componer en objetos mas complejos, los cuales pueden terminar siendo composite y asi recursivamente. Cuando el codigo del cliente espere un objeto primitivo, tambien puede tomar un objeto compuesto.
- * Hace al cliente simple. Los clientes pueden tratar estructuras composite y estructuras individuales uniformemente. Los clientes no saben (y no deberia importarles) si estan manipulando un componente Leaf o un Composite. Simplifica el codigo cliente porque evita tener que escribir etiquetas para funciones sobre clases que definen la composicion.
- * Facilita la posibilidad de agregar nuevos tipos de componentes. Componentes nuevos trabajan automaticamente con las estructuras existentes y codigo cliente. Los clientes no necesitan ser cambiados por una clase de componente nueva.
- * Puede hacer tu diseño muy general. La desventaja de que sea tan facil agregar nuevos componentes es que hace mas dificil el restringir los componentes del composite. A veces quieres que el composite tenga solo ciertos componentes. Con Composite, no puedes depender del sistema de tipado para que se fuerce esas restricciones por vos. Vas a tener que usar chequeos de run-time en cambio.

• Decorator/Wrapper

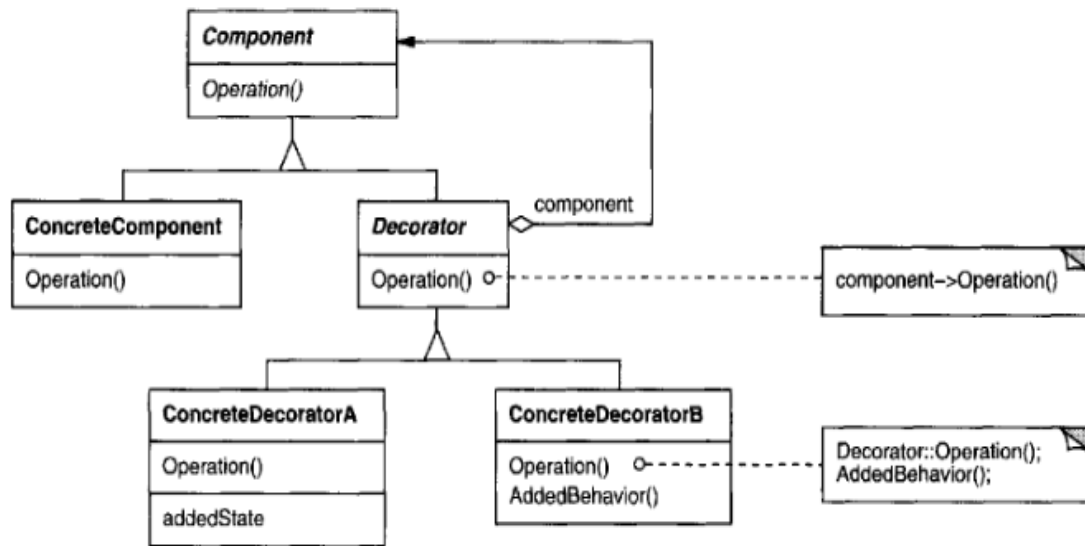
– Intent

- * Agrega responsabilidades extra a un objeto dinamicamente. Los Decorators proveen una alternativa flexible a subclasificar para extender funcionalidad.

– Applicability

- * Para agregar responsabilidades a objetos individuales dinamica y transparentemente, sin afectar otros objetos.
- * Para responsabilidades que puedan ser sacadas.
- * Para cuando la extension subclasificando es impractica. A veces, un gran numero de extensiones independientes son posibles y esto produce una explosion de subclases para hacer cada combinacion. O una definicion de clase podria estar oculta o de otra forma que no sea posible subclasificarla.

Structure



– Participants

- * **Component**
 - Define la interfaz para objetos que puedan tener responsabilidades agregadas a estos dinamicamente.
- * **ConcreteComponent**
 - Define un objeto al cual se le pueden agregar responsabilidades adicionales.
- * **Decorator**
 - Mantiene una referencia a un objeto Component y define una interfaz que corresponde a la interfaz del componente.
- * **ConcreteDecorator**
 - Agrega las responsabilidades al componente.

– Collaborations

- * El Decorator envia pedidos a su objeto Component. Tambien puede opcionalmente hacer operaciones adicionales antes y despues de enviar el pedido.

– Consequences

- * Mas flexibilidad que herencia estatica.
- * Evita clases **feature-leaden** en la jerarquia.
- * Un Decorator y su Component no son identicos.
- * Muchos objetos pequeños.

• Adapter/Wrapper

– Intent

- * Convertir la interfaz de una clase en otra interfaz que los clientes esperan. El adapter permite que clases trabajen juntas que no podrian hacerlo de otra forma debido a las interfaces incompatibles.

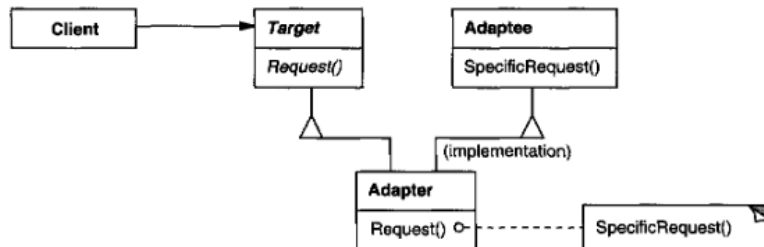
– Applicability

- * Cuando quieras usar una clase existente pero su interfaz no coincide con la que necesitas

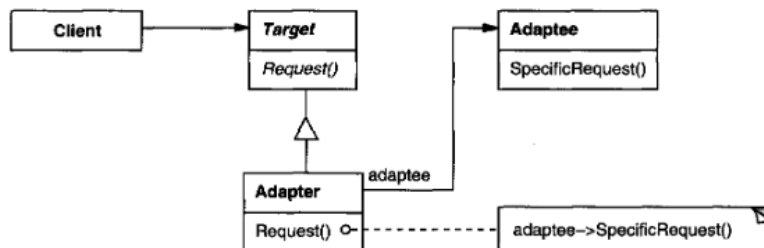
- * Cuando quieras crear una clase reusable que coopera con clases no relacionados o que todavía no sabes como van a ser, las clases no necesariamente tienen interfaces compatibles.
- * (object adapter only) Cuando necesitas usar varias subclases preexistentes pero es impracticable adaptar la interfaz subclassificando cada una. Un object adapter puede adaptar la interfaz de la clase padre de este.

Structure

A class adapter uses multiple inheritance to adapt one interface to another:



An object adapter relies on object composition:



– Participants

- * **Target**
 - Define el dominio específico de la interfaz que el Client usa.
- * **Client**
 - Colabora con los objetos que conforman la interfaz del Target.
- * **Adaptee**
 - Define una interfaz existente que necesita ser adaptada.
- * **Adapter**
 - Adapta la interfaz del Adaptee a la interfaz del target.

– Collaborations

- * Los Clients llaman a operaciones en la instancia del Adapter. Por esto el adapter llama a las operaciones del Adaptee para llevar a cabo el pedido inicial.

– Consequences

- * Class adapters y Object adapters tienen distintos pros y contras.
- * Un class adapter:
 - Adapta al Adaptee al Target comprometiéndose con una clase Adaptee concreta. Como consecuencia, un class adapter no puede funcionar cuando se quiera adaptar a una clase y todas sus subclases.
 - El Adapter sobrescribe algunos comportamientos del Adaptee, ya que Adapter es subclase de este.
 - Introduce solo un objeto, y no necesita un puntero adicional para llegar a obtener el Adaptee.
- * Un object adapter:
 - Un solo Adapter funciona para varios Adaptees (Esto es el Adaptee por sí mismo y todas sus subclases si es que tiene). El Adapter también puede agregar funcionalidad a todos los Adaptees de una.

- Es mas difícil de sobrescribir un comportamiento del Adaptee. Requeriria subclasificar un Adaptee y hacer al Adapter referirse a la subclase en vez de al Adaptee de una.
- * Otras cosas a considerar cuando se usa el adapter pattern son:
 - **Cuanta adaptacion hace el Adapter?**
 - **Pluggable adapters:** Classes with built-in interface adaptation.
 - **Usar demasiados adapters para proveer transparencia.**

• Proxy/Surrogate

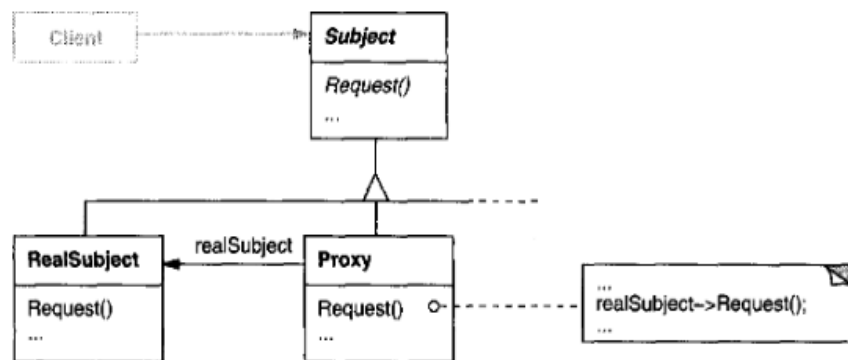
– Intent

- * Proveer un sustituto o representante de otro objeto para controlar el acceso a este.

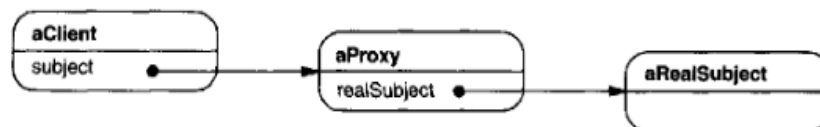
– Applicability

- * Un Proxy es aplicable cuando se necesita una referencia mas versatil o sofisticada a un objeto que un simple puntero. Algunas situaciones en las que el Proxy pattern es aplicable:
 - **Remote proxy:** Provee un representante local para un objeto en un espacio de direccion distinto.
 - **Virtual proxy:** Crea objetos costosos a demanda.
 - **Protection proxy:** Controla el acceso al objeto original.
 - **Smart reference:** Es un remplazo para un puntero pelado que hace acciones adicionales cuando un objeto es accedido.

Structure



Here's a possible object diagram of a proxy structure at run-time:



– Participants

* Proxy

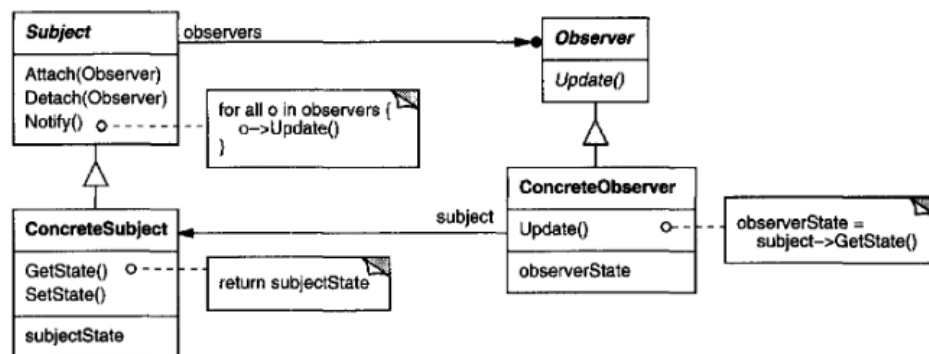
- Mantiene una referencia que permite al proxy acceso al sujeto real. Proxy puede referirse a un Subject si las interfaces de RealSubject y Subject son las mismas.
- Provee una interfaz indentica a la del Subject para que un proxy pueda ser sustituido por el objeto real.
- Controla el acceso al sujeto real y puede ser responsable de crearlo y eliminarlo.
- Otras responsabilidades dependen del tipo de Proxy:

- **Remote proxies:** Son responsables de codificar un pedido y sus argumentos y de enviar el pedido codificado al sujeto real en un espacio de direccion distinto.
- **Virtual proxies:** Pueden almacenar informacion adicional en cache del sujeto real para que puedan posponer el accederlo.
- **Protection proxies:** Chequean que el llamador tenga acceso a los permisos requeridos para realizar un pedido.
- * **Subject**
 - Define la interfaz comun de RealSubject y Proxy para que el Proxy pueda ser usado en cualquier lado donde se espera un RealSubject.
- * **RealSubject**
 - Define el objeto real que el proxy representa.
- **Collaborations**
 - * El Proxy envia (fowards) las requests al RealObject cuando sea apropiado, dependiendo el tipo de proxy.
- **Consequences**
 - * El Proxy pattern ofrece un nivel de indireccion para acceder un objeto. La indireccion adicional tiene varios usos, dependiendo del tipo de proxy:
 - Un remote proxy puede ocultar el hecho de que los objetos residen en distintas direcciones de espacio.
 - Un virtual proxy puede producir optimizaciones como crear un objeto a demanda.
 - Protection proxies y Smart references permiten agregar tareas extra cuando un objeto es accedido.
 - * Existe otra optimizacion en la que el Proxy patter puede ocultar del cliente, se llama **copy-on-write** que esta relacionado a la creacion a demanda.

• Observer/Dependents/Publish-Subscribe

- **Intent**
 - * Define una de las muchas maneras de dependencia entre objetos para que cuando un objeto cambie de estado, todos sus dependientes son notificados y actualizados automaticamente,
- **Applicability**
 - * Cuando una abstraccion tiene dos aspectos, uno dependiente del otro. Encapsular estos aspectos en objetos separados te permite cambiarlos y reusarlos independientemente.
 - * Cuando un cambio a un objeto requiere cambiar otros, o no sabes cuantos objetos tienen que ser cambiados.
 - * Cuando un objeto deberia ser capaz de notificar a otros sin hacer asunciones sobre quienes son estos objetos, En otras palabras, no quieres que estos objetos esten fuertemente entrelazados.

Structure



– Participants

* Subject

- Conoce a sus observadores. Cualquier cantidad de objetos Observer puede observar a un sujeto.
- Provee una interfaz para agregar y eliminar objetos Observer.

* Observer

- Define una interfaz para objetos que deban ser notificados de los cambios en un sujeto.

* ConcreteSubject

- Almacena el estado de interes a los objetos ConcreteObserver.
- Envia una notificacion a sus observadores cuando su estado cambie.

* ConcreteObserver

- Mantiene una referencia a un objeto ConcreteSubject.
- Almacena estados que deberian ser consistentes con los del sujeto.
- Implementa al Observer actualizando la interfaz para mantener su estado consistente con la del sujeto.

– Collaborations

- * ConcreteSubject notifica a sus observadores cuando un cambio ocurre que puede hacer que el estado de los observadores sea inconsistente con el propio.
- * Luego de ser informado del cambio en el ConcreteSubject, un objeto ConcreteObserver le puede preguntar al sujeto informacion. El ConcreteObserver usa esta informacion para reconciliar su estado con el del sujeto.

– Consequences

- * El Observer pattern te permite variar entre sujetos y observadores independientemente. Podes reusar sujetos sin reusar sus observadores y viceversa. Te permite agregar observadores si modificar el sujeto u otros observadores.
- * Acoplamiento abstracto entre Subject y Observer
- * Support for broadcast communication
- * Actualizaciones inesperadas

• State/Object for States

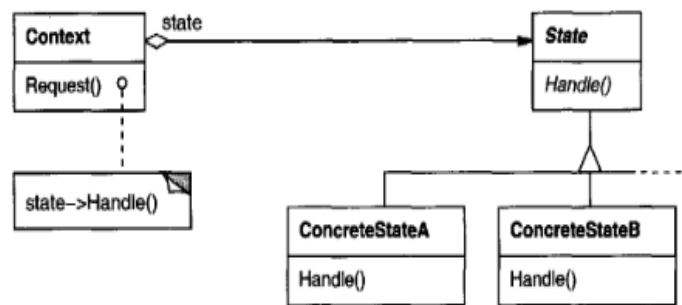
– Intent

- * Permite a un objeto alterar su comportamiento cuando su estado interno cambia. El objeto pareciera cambiar su clase.

– Applicability

- * Cuando un comportamiento de un objeto depende de su estado, y debe poder cambiar su comportamiento sobre la marcha dependiendo de su estado.
- * Cuando las operaciones tienen diversas declaraciones condicionales que dependen del estado del objeto.

Structure



– Participants

* Context

- Define la interfaz de interes para los clientes.
- Mantiene una instancia de la subclase ConcreteState que define el estado actual.

* State

- Define una interfaz para encapsular el comportamiento asociado con un estado particular del Context.

* ConcreteState subclasses

- Cada subclase implementa un comportamiento asociado con el estado del Context.

– Collaborations

- * El Context delega pedidos de estados especificos a el objeto ConcreteState.
- * Un Context puede pasarse a si mismo como argumento al objeto State encargado de manejar el pedido. Esto le permite al objeto State acceder al Context si lo necesita.
- * El Context es la interfaz primaria para los clientes. Estos pueden configurar el contexto con objetos State. Una vez que el Context esta configurado, sus clientes no pueden manejar los objetos State directamente.
- * Tanto el Context como las subclases del ConcreteState pueden decidir cual estado le sigue a otro y dentro de que circunstancia esto pasa.

– Consequences

- * **Localiza el comportamiento de un estado especifico y particiona el comportamiento para distintos estados.**
- * **Hace las transiciones de estado explicitas.**
- * **Los objetos de estado pueden ser compartidos.**

• Visitor

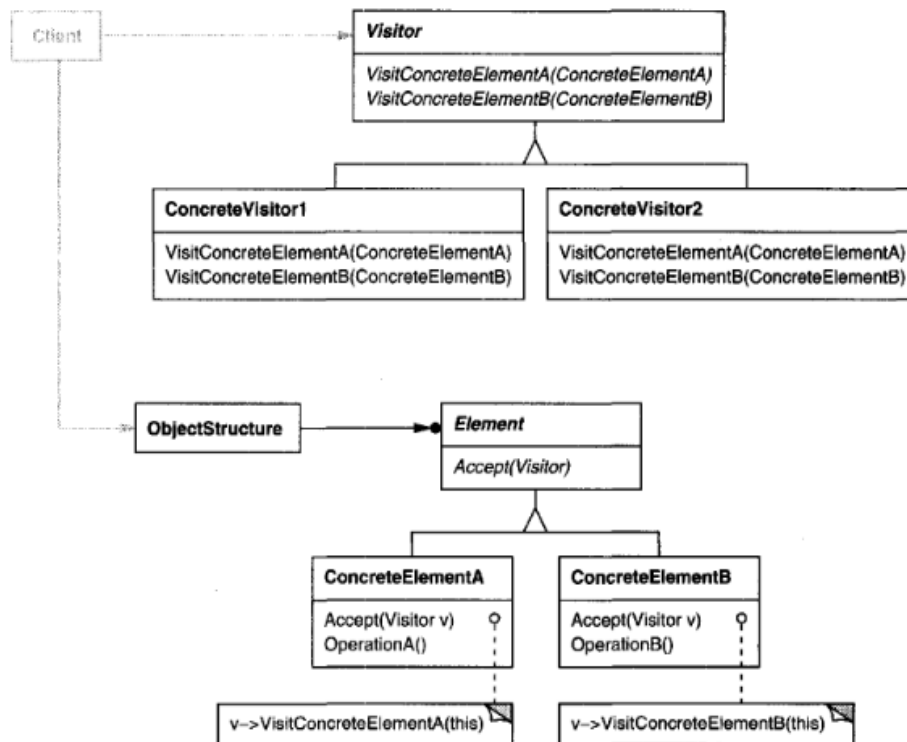
– Intent

- * Representar una operacion que ser realizara en los elementos de la estructura de un objeto. El Visitor te permite definir una operacion nueva sin cambiar las clases de los elementos con los que opera.

– Applicability

- * Cuando la estructura de un objeto contiene muchas clases de objetos con diferentes interfaces, y quieres hacer operaciones en estos objetos que dependen de sus clases concretas.
- * Cuando muchas operaciones distintas y no relacionadas se deben hacer en objetos en una estructura de objeto, pero quieres evitar contaminar sus clases con estas operaciones. El Visitor te permite mantener operaciones relacionadas juntas definiendolas en una clase. Cuando la estructura del objeto es compartida por muchas aplicaciones usa Visitor para poner estas operaciones en solamente las aplicaciones que las necesitan.
- * Cuando la definicion de las clases de la estructura del objeto raramente cambia, pero muchas veces vos quieres definir nuevas operaciones en la estructura. Cambiar la estructura del objeto de clases requiere redefinir la interfaz de todos sus Visitors, que es potencialmente costoso. Si la estructura del objeto de clases cambia a menudo, entonces es probablemente mejor definir las operaciones en esas clases directamente.

Structure



– Participants

* Visitor

- Declara una operacion Visit por cada clase de ConcreteElement en la estructura del objeto. El nombre de la operacion y su firma(? (signature) identifica la clase que envia el pedido de Visit al Visitor. Esto le permite al Visitor determinar la clase concreta de elementos siendo visitada. Entonces, el Visitor puede acceder el elemento directamente a traves de su interfaz particular.

* ConcreteVisitor

- Implementa cada operacion declarada por el Visitor. Cada operacion implementa un fragmento del algoritmo definido en la estructura de la correspondiente clase del objeto.

* Element

- Define una operacion de aceptacion que tiene al Visitor como un argumento.

* ConcreteElement

- Implementa una operacion de aceptacion que toma al Visitor como un argumento.

* ObjectStructure

- Puede enumerar sus elementos.
- Puede proveer una interfaz de alto nivel para permitirle al Visitor visitar sus elementos.
- Puede o ser un Composite o una coleccion como una lista o un set.

– Collaborations

- * Un cliente que usa el Visitor pattern debe crear un objeto ConcreteVisitor y entonces atravesar la estructura del objeto, visitando cada elemento con este.
- * Cuando un elemento es visitado, llama a la operacion del Visitor que corresonde a su clase. El elemento se manda a si mismo como un argumento de la operacion para permitirle al Visitor acceso a su estado de ser necesario.

– Consequences

- * El Visitor hace que agregar nuevas operaciones sea sencillo.
- * Un Visitor colecciona operaciones relacionadas y separa las que no.
- * Agregar nuevas clases de ConcreteElement es facil.
- * Puede visitar a traves de jerarquias de clases.
- * Pueden acumular estados.
- * Rompe encapsulamiento.

5 Papers para el final

- Ademas de 'No silver Bullet' y 'Programming as Theory Building' los siguientes papers entran:

5.1 What is Software Design? (by Jack W. Reeves)

- **Introduction**

- Jack W. Reeves da 3 perspectivas en un solo tema, el que la programacion es principalmente una actividad de diseño y que la unica y verdadera representacion del "diseño" es el codigo fuente de por si.
- En el segundo ensayo responde a las discusiones mas comunes que se ha encontrado sobre el primero.
- En el ultimo ensayo es una carta escrita al C++ journal (el que publico su primer ensayo) dando sus primeras impresiones a los temas e ideas encontrados.

- **What Is Software Desing? (Primer ensayo)**

- Comienza por el hecho de que C++ se volvio un lenguaje popular de la nada y sugiere que una posible respuesta es que este hace mas sencillo el diseño de software y del programa al mismo tiempo.
- El objetivo final de cualquier actividad ingenieril es algun tipo de documentacion final. Cuando el trabajo de diseño es completado, la documentacion de diseño es entregada al equipo de construccion. La unica documentacion de software que parece satisfacer el criterio de diseño ingenieril es el lineas de codigo fuente ([source code listings](#)).
- El articulo asume que el codigo fuente final es el diseño real de software y en base a esto examina sus consecuencias.
- Los compiladores y linkers son lo que realmente construye el software.
- El software real es increíblemente barato de construir y se vuelve mas barato a medida que las computadoras se vuelven mas rapidas.
- El software real es increíblemente caro de diseñar ya que el software es increíblemente complejo y porque en basicamente cada paso de un proyecto de software son parte del proceso de diseño.
- Programar es una actividad de diseño, un buen proceso de diseño reconoce esto y no duda en codear cuando tenga sentido hacerlo.
- Codear tiene sentido hacerlo mas sucesivamente de lo que se cree, ya que en este proceso se suelen encontrar errores o cosas que se pasaron por alto antes de llegar a casi el final del diseño terminado, lo cual hace que el diseño sea mejor al final.
- Es mas facil y barata la tecnica de build/test que el intentar probar y validar el software con procedimientos formales.
- Testing y debugging son actividades de diseño. Son el equivalente en el campo ingenieril a los procesos de validacion y refinamiento. Un buen proceso de diseño no intenta acortar los pasos.
- Hay otras actividades de diseño (top level design, module design, structural design, architectural design, etc.). Un buen proceso de diseño y deliberadamente incluye los pasos.
- Todas las actividades de diseño interactuan entre si. Un buen proceso de diseño permite el cambio, a veces radicalmente, como lo presentan por necesidad varios pasos del proceso.
- Muchas notaciones en el diseño de software son potencialmente utiles. No son diseño de software, si no, documentacion auxiliar y herramientas que ayudan al proceso de diseño.

- El desarrollo de software es mas una artesanía (*craft*) que una disciplina de ingeniería. Principalmente por la falta de rigor en el proceso critico de validar y mejorar un diseño.
- Avances reales en el desarrollo de software dependen en avances en técnicas de programación, las cuales significan avances en lenguajes de programación.
- **Punto a destacar que me llamo la atención:** "Las herramientas de desarrollo idealmente podrian procesar el código fuente y generar documentación auxiliar" Esto me hace acordar a que en Smalltalk basicamente la única documentación que precisas es el código en si, ya que te permite mediante nombres de metodos y clases entender que hace cada cosa sin necesidad de material extra.

- **What Is Software Design: 13 years later**

- Hay una wiki page sobre el primer artículo, por lo que este se canso de tratar con gente que es completamente incapaz de superar sus ideas pasadas (*pre-conceptions*).
- **Primer comentario a refutar**
 - * **"Si el código fuente es el diseño entonces los programadores son diseñadores; lo cual es falso, entonces el código fuente no es diseño"**
 - * Partio de la hipótesis de que el código fuente es el diseño y a partir de ahí pudo conseguir explicaciones mucho mejores a numerosos hechos observados.
- **Segundo comentario**
 - * **Se confunde mucho el concepto de diseño como proceso contra diseño como producto.**
 - * Se necesitan buenas arquitecturas (top level design), buenas abstracciones (class design), y buenas implementaciones (low level design).
 - * No existe la forma correcta de diseñar, ni en forma ni en cantidad de esfuerzo.
 - * El código fuente puede ser el mas importante de los elementos diseñados pero no es ni cerca el único necesario.
 - * Los programadores poco experimentados deberian aplicar todos los pasos intermedios de diseño para compensar la falta de experiencia y talento del programador promedio.
- **Tercero**
 - * **Conflicto con el concepto de que el objetivo del esfuerzo ingenieril es un tipo de documentación.**
 - * Esto es cualquier cosa, lo justifica diciendo que puede haber proyectos que no tengan documentación, pero son minoria, y generalmente son proyectos de grupos pequeños o incluso individuales
- **Cuarto**
 - * **El código fuente es de muy alto nivel para ser un diseño y que el diseño en realidad es lo que sale del compilador**
 - * Hay una cierta cantidad de flexibilidad del compilador en determinar el "como" del objeto codeado, pero no hay nada de creatividad involucrada.

- **Letter to the Editor**

- Habla de que llego a la conclusion de que no entendemos que es el diseño de software. No declara que su punto de vista es el correcto, pero lo encontro bastante util para explicar por que algunas cosas funcionan y otras no. Hay una gran diferencia entre estar haciendo un diseño de software y lo que un producto terminado de diseño de software es realmente.
- El refinamiento de todos los aspectos del diseño es un proceso que deberia suceder a lo largo del ciclo de desarrollo.
- Deberian haber dos cambios en percepcion. Primero, tendríamos que reconocer que los resultados de pasos de diseño tempranos no son un diseño de software completo, como los primeros bocetos de un puente no son el diseño final del mismo. Segundo, que el pensamiento del diseño se captura usando una notacion que es el esqueleto del diseño de software. Esto quiere decir usando un lenguaje de programación.
- Mejor sea el diseño temprano, menos trabajo es necesario refinarlo luego.

- Uno de los problemas con la vision de "Code as design" es que los lenguajes de programacion tienen serias debilidades como herramientas para expresar ciertos aspectos de un diseño de software. La informacion esta en el codigo, pero es muy dificil de sacarlo para que sea entendible por humanos. El segundo problema es similar, va a haber informacion del espacio del problema que fue al diseño de software pero que no puede ser reconstruida del diseño de software de por si. Por ambos de estos problemas se concluye que la documentacion auxiliar es importante casi tanto como en cualquier otra disciplina ingenieril.

5.2 The Early History of Smalltalk (by Alan C. Kay)

- Resumen de **resumen** sacado de CUBA-Wiki.
- Hay un tipo que lo leyo y esta **aca**.
- **1960-66: Early OOP and Other Formative Ideas of The Sixties**
 - POO vino de dos motivaciones: Querer eliminar la delegacion de tareas (**assignment**) y encontrar un mejor esquema de modelado para sistemas complejos que involucren el ocultamiento de detalles.
 - **Sketchpad and Simula**
 - * Sketchpad fue la invencion de los modernos graficos interactivos de las computadoras, las cosas eran descritas por un "master drawing" que podia producir "instance drawings".
 - * El sistema de alocacion de almacenamiento del programa era muy similar al de Sketchpad para Simula, este fue un lenguaje de procesamiento para controlar objetos del tipo de Sketchpad, teniendo, sin embargo, mas flexibilidad que limitaciones.
- **1967-69: The FLEX Machine, a First Attempt at an OOP-Based Personal Computer**
 - La maquina FLEX fue una colaboracion entre Ed Cheandle y el. Elijieron EULER como su lenguaje. Necesitaban concesiones de sintaxis, y sabian que la semantica de lo que ahora es conocido como el lenguaje FLEX debia ser influenciado mas por Simula que por EULER.
 - **Doug Engelbart and NLS**
 - * La nocion de Doug era que el destino de los oNLine Systems (NLS) era que "Va a aumentar el intelecto humano" mediante la navegacion en un espacio conceptual.
 - * Este sistema podia hacer hypertext (que es el texto de un link o un texto con hyperlinks), graficos, paneles multiples, command input, trabajo interactivo colaborativo, etc.
 - * Su interfaz de usuario era parametrica y podia ser llenada por el usuario final en la forma de "grammar of interaction" dado en su compilador-compilador TreeMeta. Estas gramaticas forzaban al usuario a estar en un estado del sistema que requeria salir antes de que cualquier nueva interaccion se pudiera hacer.
 - **FLEX Machine Concepts**
 - * Un descriptor de FLEX contenia dos punteros: Uno al "master" del objeto y el segundo para las instancias del objeto.
 - * Los objetos son una especie de mapeo donde sus valores son sus comportamientos.
 - **Dynabook**
 - * Una combinacion de sucesos se combinaron para llegar a formar una imagen de como seria una computadora personal: the Dynabook.
 - * El concepto fue creado 2 años antes de que se fundara Xerox PARC, pero estas ideas llevaron al desarrollo de Xerox Alto prototype, el cual implementaba los conceptos de GUI (graphical user interface). El software utilizado en esta investigacion fue Smalltalk, que luego este obtuvo vida propia independientemente del concepto de Dynabook.
 - **LISP and it's Inspiration for Smalltalk**
 - * Por la dificultad del lenguaje, cuando Alan Kay por fin se sentia que lo comprendia completamente, le comenzaron a intrigar cuestiones de como se utilizaba LISP en la cotidianeidad, estas dudas le inspiraron en la formacion de Smalltalk diciendo "Toma lo mas dificil y profundo que necesites hacer, hazelo espectacular y despues construi cada cosa facil de el".
- **1970-72: Xerox PARC: KiddiKomp, miniCOM, and Smalltalk-71**

– Simulation LOGO

- * Buscaba una nueva version de KiddiKomp que pueda ser hecha en una cantidad suficiente para hacer experimentos para desarrollar el diseño de la interfaz de usuario para una eventual notebook. Inspirado de POO y enfoques de patrones dirigidos (*pattern-directed approaches*) diseño un lenguaje llamado "Simulation LOGO" o SLOGO, usaria una grueso mapeado de bits de display (*coarse bit-map display*) y la tableta de goma de la FLEX machine como dispositivo apuntador (*and the FLEX machine rubber tablet as pointing device*).

– LISP and It's Inspiration for the KiddiKomp

- * Escribir varias soluciones de forma declarativa y tambien que sean el programa es parte de la atraccion y el encanto de este tipo de lenguaje. Cualquier herramienta para chicos/as deberia tener muy buenos patrones de pensamiento y una profunda belleza en ellos (*built-in*).

– MiniCOM and the Birth of Smalltalk

- * Refino la idea del KiddiKomp en un diseño mejor llamado miniCOM. Esta usaria un lenguaje que ahora llama "Smalltalk" como en "programar deberia ser una cuestion de ..." y "los niños/as deberian programar en ..."

– Smalltalk-71

- * Era una especie de parser con object-attachment que ejecutaba tokens directamente.
- * Un problema traído de LISP le hizo darse cuenta que necesitabamos control total sobre lo que se enviaba en un mensaje. En particular, cuando y en que ambiente las expresiones eran evaluadas? Dave Fisher introdujo el concepto de **reflective design** para resolver esto: Asociar el link del estado global con expresiones y funciones que se evaluarán mas tarde para que las variables libres referenciadas sean las que realmente son implicadas por el lenguaje estatico.

– Improvements on the display

- * Una de las preocupaciones era el tamaño del bit-map display, era difícil de tener informacion suficiente en la pantalla.
- * Con la contribucion de varios factores se consiguio la primera ventana sobrepuesta aparecer.

– Advancements on Education

- * Preocupado por que el directo enfoque simbolico tomado por FLEX, LOGO (y Smalltak) fuese difícil de procesar para los niños/as Alan se puso a investigar formas de hacerlo mas facil y se encontro con Rovner's AMBIT-G, el cual se veia prometedor, era una especie de visual del tipo SNOBOL y las ideas de pattern matching se veian como que podria utilizarse para el esquema mas PLANNERlike que estaba queriendo.

● 1972-76: The first real Smalltalk (-72), its birth, applications, and improvements

– Early Versions

- * Los simbolos eran byte-coded y los receptores de return-values de un envio eran simtricos (receipt podria ser como un enlace de parametro). Esto era particularmente util para el retorno de varios parametros. Esto fue abandonado para que sea mas un estilo de retorno funcional orientado en expresiones.
- * Ingalls estaba fascinado con un sistema que "siempre corriese" y en los siguientes 10 años hizo por lo menos 80 publicaciones importantes de distintos sabores de Smalltalk.

– The Iterim Dynabook

- * En Abril de 1973, el primer Interin Dynabook BILBO fue creado. Hubo muchas obstrucciones por parte de la prensa y un ejecutivo que quiso matar el proyecto pero se logro finalmente conseguirlo.

– Smalltalk's Main Ideas

1. Todo es un objeto.
2. Los objetos se comunican a traves del envio y recepcion de mensajes (en terminos de objetos).
3. Los objetos tienen su propia memoria (en terminos de objetos).
4. Todo objeto es una instancia de una clase (la cual debe ser un objeto).
5. La clase contiene el comportamiento compartido de sus instancias (en la forma de objetos en una lista de programa).

- 6. Para evaluar una lista de programa, el control es pasado al primer objeto y el resto es tratado como su mensaje.
- * Los **primeros tres principios** son lo que los objetos "son sobre", como estos son vistos y usados desde "afuera".
- * Los **ultimos tres principios** de los objetos fueron retocados por dentro en cada version de Smalltalk.

– Development of the Smalltalk-72 System and Applications

- * Ventanas sobrepuestas fue el primer proyecto conseguido.
- * Una de las siguientes clases para ser implementadas era una version orientada a objetos de la LOGO turtle. Esto podria hacer varias instancias que podrian ser usadas para dibujar y como un tipo de valor para transformaciones graficas.
- * miniMOUSE es el primer real WYSIWYG (what you see is what you get) editor de galeria.
- * Findit interfaz que usaba una analogia de clases en sus instancias para hacer pedidos de recuperacion.
- * OPUS, el primer sistema de anotador de puntaje de tiempo real.
- * SHAZAM un maravilloso simple sistema capaz de crear simplea animaciones.
- * PYGMALION, un ensayo entrando en programacion iconica. Uno programaba mostrandole al sistema como los cambios deberian hacerse, como uno le mostraria a otro programador en una pizarra como los cambios deberian hacerse.

– The Evolution of Smalltalk-72: Smalltalk-74

- * Incorporo importantes mejoras, dando un objeto real al "mensajero", diccionarios de mensajes para clases (lo cual permitio dar un paso a clases de objetos reales), Diana Merry's bitblt (el ahora famoso operador de graficos 2D para graficos bitmap) y una mejor, mas general interfaz de ventana.
- * El problema ahora es el purgamiento de conjuntos de objetos no funcionales para hacer espacio a los que se necesitan.
- * La clave para el diseño era el esquema de checkpoints que se genero. Esto aseguraba que siempre hubiese una imagen recuperable de no mas de unos segundos de hecha.

– "Object-oriented" Style

- * Lo ultimo que quieres que cualquier programador haga es tocar el estado interno aunque este presentado figurativamente. En cambio, los objetos deberian estar presentados como sitios de un comportamiento de un nivel mas alto mas apropiados para el uso de componentes dinamicos.
- * Cuando le das a alguien una estructura, rara vez quieres que ellos tengan privilegios ilimitados a estos. Para esto tenes que hacer que todos los objetos sean importantes y proteger a todos.

– Smalltalk and Children

- * No es suficiente solo haber aprendido a leer y escribir. Necesitas literatura que cultive ideas. Decidieron que tendrian que enseñar diseño, introduciendo un intermediario entre las ideas vagas sobre el problema y la detallada escritura y debugging que se debia hacer para hacer Smalltalk correr: **Design Templates**.
- * **Lo que es dificil es determinar que ideas mostrar y que tan profundamente deberian penetrar a un nivel de desarrollo del niño.**
- * Las herramientas proveen un camino, contexto, y casi una excusa para desarrollar iluminacion, pero ninguna la contiene o puede desprenderla.
- * El conocimiento esta en su estado menos interesante cuando se esta aprendiendo.
- * Uno de los caminos utiles es fluidez que es parte del proceso de construccion de estructuras mentales que desaparecen la interpretacion de las representaciones.
- * El segundo camino es tomar el conocimiento como una metafora que puede iluminar otras areas.
- * El "truco" es conseguir la fluidez y profundidad mientras que se construyen relaciones con otro que sea fluido y con conocimiento profundo.

• 1976-80: The first modern Smalltalk (-76), its birth, applications, and improvements

- Se queria fuertemente un mecanismo de herencia real.
- Dan debia encontrar una mejor manera que Simula para el tiempo de compilada. Era momento para hacer un buen uso de la idea de "todo es un objeto" que incluya todos los objetos de "sistemas" internos como "activation records" etc.

- La sintaxis flexible de Smalltalks anteriores era muy flexible, y este nivel de extensibilidad no era deseable.
- A Dan se le ocurrió una sintaxis keyword/operator que era bastante flexible pero permitía que el lenguaje se lea sin ambigüedad para el humano y la máquina. Esto le permitió a una máquina del tipo FLEX un compilador byte-code y un interpretador eficiente definibles que corrían a una velocidad 180 veces superior al anterior interpretador directo.

- **Inheritance**

- * A Dan Ingalls se le ocurrió un esquema que era como Simula en su semántica pero que podía ser modificado incrementalmente *in situ* sobre la marcha para estar en sintonía con los objetivos de la interacción cercana. Alan no estaba muy emocionado al respecto porque creía que necesitaban una mejor teoría de herencia.
- * Herencia e instanciación (que es un tipo de herencia) confunde ambas pragmáticas (como factorizar código para ahorrar espacio) y semánticas (se usan muchas tareas como: especialización, generalización, especialización, etc.).

- **The Smalltalk User Interface**

- * El enfoque de la interfaz eran los niños/as. Esto llevó a pasar de "acceso a funcionalidad" a "ambiente en donde los usuarios aprendan haciendo".

- **Smalltalk-76**

- * Uno de las clases más icónicas de este fue la Ventana. Con esta nueva versión, el código es expresado como objetivos de conseguir para otros objetos (o sí mismo).
- * Más tarde se creó el Smalltalk Simkit, que es Smalltalk-76 con un abundante sistema hecho a medida para usuarios adultos no expertos. Su interfaz era una simulación de herramienta de trabajo que puedan ayudar a los ejecutivos en armar simulaciones dinámicas.
- * El sistema de por sí era correcto pero debía estar completamente cerrado de el propio Smalltalk, particularmente respecto al error con los mensajes.

- **1980-83: The release version of Smalltalk (-80)**

- Alan estaba contento por la pulcritud y elegancia de la concepción de Smalltalk, pero seguía queriendo que sean "playstations" en vez de "workstations".
- Adele decidió conducir el proceso de documentación y publicación de un nuevo Smalltalk que pueda ser distribuido mundialmente sin importar el hardware objetivo.
- Se introdujeron metaclases lo cual confundió a Alan ya que como dijo Peter en su momento "Las metaclases probaron ser más confusas que útiles".
- Cuando uno puede ver una instancia desde múltiples perspectivas, incluso una "semi-metaclass" como Class Class y Class Object no son realmente necesarias.
- Lo anterior junto a otras ideas valiosas estaban disponibles para desarrollar pero no fue adoptado. Su suposición es que Smalltalk ya se había movido a la fase final de desarrollo.

- **Coda**

- * El hardware que se les presentó como ya dado y depende del software hacerlo parecer razonable. Esto causó que el hacer técnicas de bajo nivel y excesiva optimización retuvieran el progreso en el diseño de software. Como decía Bob Barton "Los programadores de sistemas son los sumos sacerdotes de un culto bajo".
- * Todas las operaciones elementales que tenían que suceder rápido tenían que ser completadas sin bajarle la velocidad a la máquina.
- * Si la trampa decide que los objetos en cuestión son muy complicados para la ALU el HW tenía que dinámicamente encontrar un método que pueda manejar a los objetos.
- * El objetivo principal de POO es no tener que preocuparse por lo que está dentro de un objeto.
- * Uno de los esquemas de late-binding más lindo con el que se está experimentando es metaobject protocol. Esto es un descendiente directo de direcciones similares de los pasados Simula, FLEX, CDL, Smalltalk, and Actors.
- * Otro esquema de late-binding es alejarse del protocolo directo de matcheo cuando un objeto nuevo aparece en un sistema de objetos.
- * Una de las predicciones que hace es que los objetos van a llevar mucha más información sobre ellos en una forma que permita el docking inferencial.

- * Uno de los problemas que tiene el siglo XX es que al ser que la tecnología se volvió muy fácil ahora se hacen cosas porque puedes hacer nada más.
- * A pesar de tener que correr en casi cualquier lado en arquitecturas de HW moribundas, Smalltalk ha proliferado sorprendentemente bien.

5.3 Preguntas y respuestas de Final (vistas en telegram)

- Mensaje de Telegram: Buenas! si a alguien le sirve en esta **charla de Hernán** hace una mención sobre complejidad esencial/accidental que se habla en el paper de Brooks (aprox. minuto 13) y sobre la decisión de tener el concepto de objeto como concepto unificador (que fue una influencia de LISP) en el minuto 32 aprox. También **esta charla** resume varias "inspiraciones" al comienzo.
- **Que diria Platon del Metamodelo? (The Early History of Smalltalk)**
 - Respuesta Telegram:
 - * Smalltalk tiene una forma de crear objetos platónica, porque tiene ideas (clases) y manifestaciones de ideas (objetos). Pero las clases son objetos entonces son como manifestaciones de la idea de ideas, entonces a Platon le parecería divertido como se autodescribe.
 - Otra:
 - * Era que le parecería un chiste.
 - GPT:
 - * Platón probablemente vería el metamodelo de Smalltalk como una manifestación de su teoría de las Formas. Así como en su filosofía las Formas son las ideas perfectas e inmutables que dan origen a las cosas concretas, el metamodelo en Smalltalk define la estructura abstracta que rige a todos los objetos.
 - * Desde su perspectiva, el metamodelo sería el "mundo inteligible" que organiza el "mundo sensible" de las instancias en el sistema. Al igual que su alegoría de la caverna, donde las sombras representan copias imperfectas de la realidad, los objetos serían manifestaciones específicas de principios abstractos definidos en el metamodelo.
- **En que cosas se inspiró Alan Kay para crear la POO? (Tecnologías + Teorías) (The Early History of Smalltalk)**
 - Simula 67, Sketchpad, Lisp, iirc(?), teorías de aprendizaje.
 - GPT:
 - * Según The Early History of Smalltalk, Alan Kay se inspiró en varias tecnologías y teorías para desarrollar la programación orientada a objetos (POO). Estas influencias se pueden clasificar en dos categorías principales: tecnologías previas y teorías conceptuales.
 - * Tecnologías que influyeron en la POO:
 1. Simula 67
 - Introdujo el concepto de clases y objetos.
 - Implementaba herencia y encapsulación para modelar sistemas complejos.
 2. Lisp
 - Mostró la importancia de un sistema basado en funciones y estructuras dinámicas.
 - Introdujo la idea de que el código y los datos pueden representarse de manera uniforme.
 3. Sketchpad (Ivan Sutherland, 1963)
 - Sistema gráfico interactivo con objetos gráficos reutilizables y jerárquicos.
 - Permitía definir y manipular formas visuales como instancias de una clase.
 4. Flex (William Newman, 1968)
 - Sistema de manipulación directa basado en la idea de "iconic programming" (programación visual).
 - Inspiró el diseño de Smalltalk en términos de interfaces gráficas y manipulación interactiva.
 5. ARPA/PARC Computing Research (Xerox PARC)
 - Trabajo en interfaces gráficas con ventanas, mouse y manipulación directa.
 - Desarrollo de hardware y software interactivo para mejorar la experiencia del usuario.
 - * Tecnologías que influyeron en la POO:

1. Biología y Teoría de los Sistemas
Kay veía los objetos como células en un organismo, comunicándose mediante mensajes en lugar de compartir estado directamente.
Inspirado en el trabajo de Maturana y Varela sobre sistemas biológicos y autopoiesis.
 2. Teoría de la Comunicación (Shannon and Weaver, 1948)
Modelos de transmisión de información influyeron en la idea de que los objetos envían mensajes en lugar de acceder directamente a datos internos.
 3. Constructivismo (Jean Piaget)
La idea de que el aprendizaje ocurre a través de la construcción activa del conocimiento.
Esto llevó a Kay a diseñar Smalltalk como un lenguaje fácil de aprender, exploratorio e interactivo, ideal para la enseñanza.
 4. Teoría del Aprendizaje de Seymour Papert (LOGO)
Inspiración en el uso de la computadora como una herramienta de exploración para niños.
Smalltalk fue pensado como un entorno donde el usuario puede experimentar y aprender progresivamente.
- * La POO en Smalltalk no solo fue una evolución técnica, sino una combinación de ideas de simulación, biología, aprendizaje y comunicación. Alan Kay no solo quería un lenguaje eficiente, sino un sistema que reflejara la manera en que los seres humanos y las máquinas podían interactuar y aprender.

• Que era Simula 67? (The Early History of Smalltalk)

– GPT:

- * Simula 67 fue un lenguaje de programación desarrollado en Noruega por Ole-Johan Dahl y Kristen Nygaard en la década de 1960. Se considera el primer lenguaje orientado a objetos, ya que introdujo el concepto de clases y objetos junto con mecanismos como la herencia y la simulación de procesos.
- * El propósito original de Simula 67 era modelar sistemas complejos mediante simulaciones, especialmente en el ámbito de la investigación operativa y la simulación de eventos discretos. Sin embargo, su impacto fue mucho mayor, ya que sentó las bases para la programación orientada a objetos (OOP), influyendo directamente en lenguajes posteriores como Smalltalk, C++, Java y otros.
- * Alan Kay, en el desarrollo de Smalltalk, se inspiró en Simula 67, pero lo llevó más allá al hacer que todo en el sistema fuera un objeto y al enfatizar el envío de mensajes entre objetos en lugar de solo la estructura de clases.

• Por que cosas esta dada la complejidad esencial del software? En cuales ayuda TDD? (No Silver Bullet)

– Esto seria Complejidad, Cambiabilidad, Conformidad e Invisibilidad.

– Respuesta Telegram:

- * Complexity: Ayuda a construir el código de a poco.
- * Conformity: Este concepto dice que "muchacha complejidad viene de la conformación a otras interfaces; esto no puede ser simplificado por ningún rediseño del software solamente", a; ir desarrollando con TDD, te vas asegurando que vas cumpliendo con todas las interfaces.
- * Changeability: Este concepto sostiene que "la entidad del software es constantemente sujeta a presión para que cambie", el TDD construye el software de manera incremental, entonces cualquier tipo de cambio puede realizarse sin problemas.
- * Invisibility: Esto está un poco tirado de los pelos, pero: Si bien TDD no nos permite "ver" el software, nos permite estar bien familiarizados con su funcionalidad, por lo que de alguna manera obtenemos la "familiaridad" que nos daría poder visualizarlo.

– Otra:

- * Medio que TDD te ayuda con todas.
- * Hernan me dijo que quería que le respondiera si no recuerdo mal **Conformity**.
- * En el fondo la complejidad va a ser la misma (la dada por la esencia). El hecho de que sea invisible no lo puedes cambiar, y el hecho de que todo el tiempo te pidan cambios tampoco. Si te puedes asegurar con TDD que el programa cumple con la interfaz.

- Otra:
 - * Me pregunto esto y respondi conformity. Me dio a entender que ayudaba mas a changeability mas que a conformity (o tal vez esperaba que le pudiese explicar bien porque conformity y no changeability).
- Otra:
 - * TDD y dije variabilidad e invisibilidad, me dijo que estaba de acuerdo, y me dijo si para mi atacaba complejidad, le dije que no sabia.
- GPT:
 - * Según No Silver Bullet de Fred Brooks, la complejidad esencial del software surge de cuatro factores principales:
 1. Complejidad del dominio
El software debe modelar problemas del mundo real, que suelen ser intrincados y llenos de excepciones.
 2. Conformidad
El software debe ajustarse a múltiples restricciones externas (regulaciones, hardware, sistemas heredados, etc.), lo que lo hace más difícil de modificar y entender.
 3. Cambio
Los requisitos evolucionan constantemente debido a nuevas necesidades, regulaciones o descubrimientos, lo que exige adaptabilidad continua.
 4. Invisibilidad
A diferencia de estructuras físicas, el software no tiene una forma visual clara, lo que dificulta su comprensión y comunicación.
 - * TDD no elimina la complejidad esencial, pero ayuda a manejar algunos aspectos:
Manejo del cambio → Los tests sirven como documentación viva, facilitando refactorizaciones seguras.
Conformidad → Asegura que el software cumpla con requisitos específicos al validar comportamientos esperados.
Invisibilidad → Proporciona una forma estructurada de definir y verificar funcionalidades, reduciendo la opacidad del código.
Sin embargo, TDD no resuelve la complejidad del dominio, ya que esta depende de la naturaleza del problema en sí.

- **Que decia Alan Kay del metamodelo de Smalltalk? (The Early History of Smalltalk)**

- Respuesta Telegram:
 - * La rta iba por el lado de que es muy complicado, si mal no recuerdo. Que era una paja mental.
 - * Muy complicado y que no mejoraba lo suficiente para que valga la pena.
- Otra:
 - * Decía que era confuso. Habla de eso cuando habla de Smalltalk 80. Dice que introdujeron las metaclasses para hacer un poco más fácil la inicialización de instancias, pero que eso ya lo hacía bastante bien el Smalltalk 76. Un tal Peter Deutsch dijo que las metaclasses eran confusas para muchos usuarios, y en el balance eran más confusas que valiosas, y Alan Kay lo bancó en esa.

- **Decime cosas interesantes que dice Brooks sobre su paper (ok Hernan(?)) (No Silver Bullet)**

- **Persona que rindio: "Le conte que trabajaba, y me pregunto como atacabamos algunas de las dificultades esenciales en mi trabajo"**

- **Que conexiones encontras entre Brooks y Naur? (No Silver Bullet y Programming as Theory Building)**

- Respuesta de alguien:
 - * Ambos ponen en un lugar de mucha importancia y jerarquia al programador. Naur (Programming as Theory Building) lo hace literalmente, argumentando que el trabajo de un programador es construir una teoría (y que el código es una consecuencia de esa teoría) y como transferir esa teoría de un programador a otro es muy costoso, se debe tratar a los programadores como cualquier otro profesional dándole una oficina y responsabilidades (etc). Por otro lado Brooks entiende que el desarrollo del software tiene dificultades esenciales (para las cuales parece no haber atajos ni soluciones) y como el programador es el que debe afrontar esas dificultades, indirectamente le está dando un lugar de jerarquía a los programadores.

* Básicamente los dos dicen que finalmente la responsabilidad va a recaer sobre los programadores.

• **Que dice Alan Kay sobre Lisp? Que era lo bueno/malo de Lisp segun él? (The Early History of Smalltalk)**

– Respuesta dada (no rindio):

- * Lo bueno: Fijate las cosas que copió de lisp a smalltalk.
- * Lo malo: Fijate las cosas que tiene smalltalk que no tiene lisp.
- * El entorno metacircular y los closures por ej.
- * Medio que con smalltalk lleva el entorno meta a otro nivel, porque ya no es un REPL, sino que todo el entorno está vivo y ejecutando todo el tiempo

– Otra (no rindio):

- * Lo de la metaestructura y el diseño reflexivo seguro. Después en varios lugares hace planteos sobre la evaluación de expresiones en LISP (y un guiño al tema de las responsabilidades en el envío de mensajes).
- * Está inspirado en esto (de "Design Principles Behind Smalltalk")

Metáfora Uniforme: *Un lenguaje debería ser diseñado alrededor de una metáfora poderosa que pueda ser aplicada uniformemente en todas las áreas.*

Ejemplos de éxitos en esta área incluyen a LISP, que está construido sobre el modelo de listas enlazadas; APL, que está construido sobre el modelo de arreglos; y Smalltalk, que está construido sobre el modelo de objetos que se comunican. En cada caso, las aplicaciones grandes son vistas en la misma manera en que las unidades fundamentales de las cuales el sistema está construido. Especialmente en Smalltalk, la interacción entre los objetos más primitivos es vista en la misma manera que la interacción de más alto nivel entre la computadora y su usuario. Todo objeto en Smalltalk, hasta un humilde entero, tiene un conjunto de mensajes, un protocolo, que define la comunicación explícita a la que ese objeto puede responder. Internamente, los objetos pueden tener almacenamiento local y acceso a otra información compartida que comprenden el contexto implícito de toda comunicación. Por ejemplo, el mensaje + 5 (sume cinco) lleva una suposición implícita de que el sumando es el número que recibe el mensaje.

– Otra (rindio):

- * Sintaxis complicada y modelo poco coherente al plantear todo como formula pero utilizar formulas especiales como base

• **Que estructuras de Lisp estan agregadas como parte del lenguaje? Y por que cond esta de esta manera? (The Early History of Smalltalk)**

– Rta:

- * + Disculpen, las estructuras agregadas son cond, quotes, y expresiones lambda. Pero por qué el cond esta de esta manera no me queda claro...
- * - Porque decidieron hacerlo asi. Osea podrías hacerlo como una función de tres parámetros pero si quieres que evalúe solo una rama tenés que meterlo en una lambda para que no evalúe. Creo que lisp es call by value si no me acuerdo mal.

– GPT

- * Según el paper "The Early History of Smalltalk" de Alan Kay, en Lisp había ciertas formas especiales (special forms) que no se comportaban como funciones. Estas eran necesarias porque evaluaban sus argumentos de manera diferente a las funciones normales. Algunas de las formas especiales mencionadas en el contexto del paper incluyen:

QUOTE – Evita la evaluación de un argumento, permitiendo tratar símbolos y listas como datos en lugar de expresiones a evaluar.

COND – Estructura de control condicional (similar a if-else pero con múltiples condiciones).

LAMBDA – Define funciones anónimas sin evaluarlas inmediatamente.

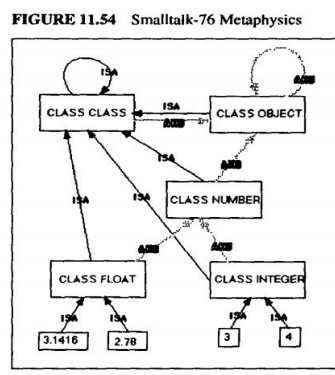
DEFINE – Permite definir variables y funciones globales.

SETQ – Asigna valores a variables sin evaluarlas como una función normal.

- * Estas formas especiales eran necesarias porque Lisp evaluaba todos los argumentos de las funciones antes de ejecutarlas, y algunas operaciones requerían un control más explícito sobre la evaluación. Esto contrasta con Smalltalk, que manejaba el control de flujo a través del paso de mensajes y bloques de código en lugar de formas especiales.

- **Hablame un toque sobre el metamodelo de Smalltalk (The Early History of Smalltalk)**

- Imagen de metamodelo:



- **Criticas a Smalltalk (The Early History of Smalltalk)**

- Rta:

- * Hablé de la herencia y las metaclases (class class, class object).

- **Extra:**

- De una persona de telegram:

- * Si te sirve en un toque te paso las que me hice yo, no será oficial pero capaz te sirve:
- * Qué elementos utiliza para comparar la ingeniería de software con otras?
- * Cómo es la comparación que hace entre la metodología de desarrollo imperante en ese momento (design documentation -¿ manufacturing team) y su idea de la fuerte relación entre el diseño de alto y bajo nivel (influencia mutua entre ambos)?
- * Qué lugar le da al testeo y debuggeo al hacer la comparación con otras ingenierías?
- * Qué relación le encontrás con No Silver Bullet?
- * Qué rol dice que cumple toda otra documentación que no sea el código fuente?
- * Qué característica dice que es importante tener en un lenguaje con respecto al diseño?

- Otra:

- * Creo que a veces pregunta detalles chiquitos sobre el texto de Kay para evaluar si lo leíste. A mi no me bajó nota por no saberla porque cuando le hablé de otras cosas cité otros detalles del texto.
- * Vi que a otro le habían preguntado sobre temas pedagógicos (básicamente que piaget decía que el pensamiento icónico era más fácil que el simbólico y se aprendía antes) y que eso justificaba ir más hacia smalltalk e interfaces gráficas amigables.
- * Yo tampoco tenía muy claro que era el metamodelo. me preguntó sobre eso y dije algunas cosas sobre la herencia, pero se refiere específicamente al tema de que hay una clase clase y una clase objeto.

- Otra:

- * La base de la nota era como me habia ido en los parciales/tps. De ahí redondeo dos puntos para arriba porque fui preparado. Mi cursada igual fue pre-pandemia, así que puede que haya cambiado (**Sigue siendo así**).

- Mucha suerte a todo el que le llegue este resumen <3. No confirmo que este correcto ya que esta hecho para mi pero se hizo lo que se pudo (?).